

Aufgabe 2: Simultane Labyrinth

Teilname-ID: 74314

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

26. Dezember 2024

Inhaltsverzeichnis

1 Lösungsidee	1
1.1 4-dimensionalen Labyrinth	2
1.2 Punktesystem	2
1.2.1 Beispiel	2
1.3 Fortschrittsregel	3
1.4 Wipeoutregel	3
1.5 Zusatzherausforderung: Funktion verallgemeinern	3
2 Umsetzung	4
2.1 main	4
2.2 Read_labyrinth	4
2.3 Solve_labyrinth	5
2.4 Solve_duo_labyrinth	6
2.5 Umsetzung des solve_n-labyrinth	7
3 Analyse des Programms	8
3.1 read_labyrinth	8
3.2 solve_labyrinth	8
3.3 Solve_duo_labyrinth	8
3.4 main	9
3.5 Einstufung des Problems	9
3.6 Analyse des verallgemeinerten Labyrinths ($2 \cdot k$ -dimensionales Labyrinth)	9
4 Einstufung des Problems:	9
5 Beispiele	10
6 Quellcode	20
6.1 Huffman-Codierung	25

1 Lösungsidee

Als Lösungsidee wird zunächst einmal bei beiden gegebenen Labyrinth ein Breadth-First Search Algorithmus ausgeführt, um den kürzesten Weg zum Ziel bei beiden auszurechnen. Dabei werden alle Gruben wie eine Blockade behandelt (ein unpassbarer Weg), die Wände als Sperre und die Lösungen als Koordinaten sowie Wegbeschreibungen zurückgegeben. Besuchte Koordinaten werden in einer Liste gespeichert und nicht noch einmal besucht. Es werden alle Bewegungsmöglichkeiten von einer Koordinate angeschaut, und mithilfe einer Liste wird eine Warteschlange erstellt, die alle neuen Koordinaten speichert, sowie die vergangenen Positionen und Bewegungen.

Dadurch soll einerseits geschaut werden, ob sich dieses Labyrinth lösen lässt. Gleichzeitig sollen diese Koordinaten als Anhaltspunkt für die nächste Aufgabe gelten. Falls sich ein Labyrinth nicht lösen lässt, so werden nur die Ergebnisse des anderen ausgegeben bzw. falls sich beide nicht lösen lassen, keine ausgegeben.

Anschließend werden beide Labyrinth zu einem 4-dimensionalen Labyrinth kombiniert, wobei x_1 , y_1 (aus dem ersten Labyrinth) und x_2 , y_2 (aus dem zweiten Labyrinth) je als eine Dimension gelten.

1.1 4-dimensionalen Labyrinth

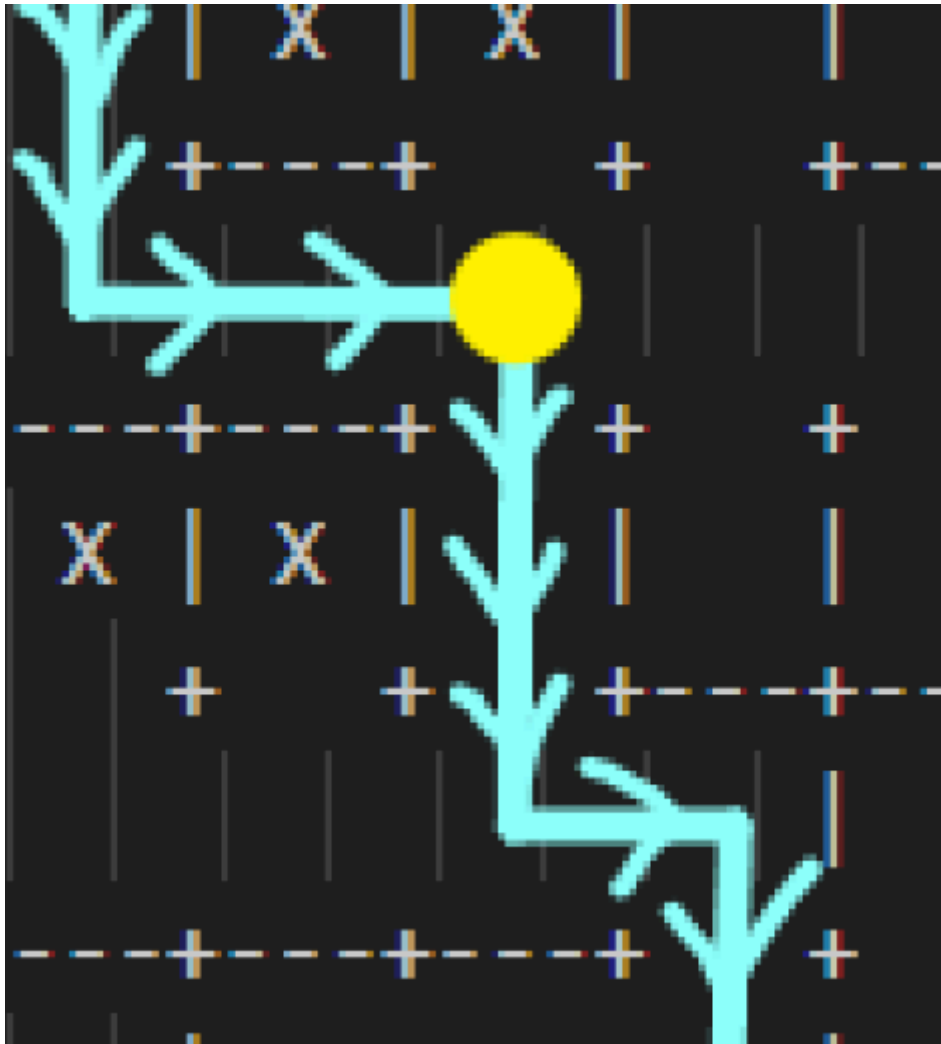
Im 4-dimensionalen Labyrinth wird anschließend wieder der Breadth-first-search Algorithmus ausgeführt. Diesmal werden die Gruben als Zurücksetzpunkte anerkannt und setzen die x- sowie y-Koordinaten des Labyrinths auf 0, 0 zurück. Wände blockieren abhängig vom Labyrinth die Charaktere und die Charaktere bleiben beim Erreichen des Ziels stehen. Besuchte Koordinaten werden als 4-dimensionale Koordinaten (x1, x2, y1, y2) bzw. als eine Gruppe gespeichert und es wird darauf geachtet, dass vergangene Positionen sich nicht wiederholen, indem überprüft wird, ob alle 4 Koordinaten - als eine Gruppe - schon bereits besucht wurden.

1.2 Punktesystem

Dank der zuvor ausgerechneten Wegbeschreibung (die sich durch das einzelne Lösen des Labyrinths ergibt), soll hier diese Wegbeschreibung als Orientierungspunkt verwendet werden, um die Berechnungszeiten massiv zu verringern. Dabei entsteht ein Punktesystem, das die Entfernung zum Ziel anzeigt. Jede Koordinate bzw. jeder Weg erhält eine Punktzahl, die aus der Summe der Anzahl der richtigen Wegentscheidungen beider Labyrinth besteht (als richtige Bewegung wird das Folgen der ausgerechneten Wegbeschreibung bezeichnet). D.h. Für jede richtige Bewegung erhält man einen Punkt. Bewegen sich beide Labyrinth in die richtige Richtung, so erhält man insgesamt 2 Punkte. Wenn eine Koordinate von ihrem Lösungsweg abweicht, so bleibt die Punktzahl für dieses Labyrinth unverändert. Wenn eine Koordinate ihren Lösungsweg zurückläuft, so wird bei diesem Labyrinth ein Punkt abgezogen. Hinzu kommt noch, dass eine Rekordpunktzahl hinzugefügt wird. Koordinaten, die eine neue Rekordpunktzahl erreichen, werden als Rekord gespeichert und als Referenzpunkt für andere Koordinaten verwendet. Fällt eine Koordinate + Pufferpunkte unter diese Rekordpunktzahl, so werden diese Koordinaten für den weiteren Verlauf nicht mehr verwendet. Pufferpunkte sind Gratispunkte, die alle Koordinaten bekommen, damit neue Möglichkeiten und evtl. auch Wege gefunden werden, die die zukünftige Rekordpunktzahl erreichen könnten. Pufferpunkte werden bei der Rekordpunktzahl nicht berücksichtigt, da die Rekordpunktzahl nur von der tatsächlichen Punktzahl abhängig ist. Die Pufferpunkte kann man selber einstellen: Je größer die Pufferpunkte, desto kürzer wird die Wegbeschreibung zum Ziel und gleichzeitig nimmt die Berechnungsdauer zu. Dadurch soll ein Wettbewerb entstehen, welcher Weg am meisten Punkte bekommt und nicht rausfliegt. Dadurch erreicht man, dass man einen kurzen Weg erhält und gleichzeitig ein geringerer Zeitaufwand notwendig ist.

1.2.1 Beispiel

Beispiel: Im Bild unten ist der gelbe Kreis der Spieler. Die blauen Pfeile sind die zuvor berechnete kürzeste Wegbeschreibung. Orte, die mit einem X markiert sind, stellen die Gruben dar und die Wände wurden kenntlich mit einer weißen Linie gekennzeichnet. Bewegt sich der Spieler nach unten, so erhält er zu seinem aktuellen Punktestand +1 Punkt. Bewegt sich der Spieler nach links, so zieht man von seinem aktuellen Punktestand einen Punkt ab. Bewegt sich der Spieler nach rechts, so bleibt sein Punktestand unverändert. Bewegt sich der Spieler nach oben, so fällt er in eine Grube. Da die Punktzahl den Fortschritt zum Ziel anzeigt, wird sein Punktestand auf 0 zurückgesetzt. Durch die Kombination mit dem zweiten Labyrinth, wird eine gemeinsame Punktzahl geschaffen. Fällt diese gemeinsame Punktzahl + Pufferpunkte unter die aktuelle Rekordpunktzahl, so scheiden diese Koordinaten (bzw. dieser Weg) aus und es werden keine weiteren Koordinaten von diesem Punkt aus berechnet.



1.3 Fortschrittsregel

Zusätzlich gibt es noch die Fortschrittsregel: Mind. 1 Labyrinth muss bei jeder Bewegung Punkte erzielen. Erzielt keines der beiden Labyrinth durch ihre Bewegung einen Punkt, so bringt diese Bewegung nichts -> Diese Bewegung/dieser Weg wird verworfen. Erzielt ein Labyrinth Punkte und gleichzeitig verliert das andere Labyrinth Punkte, ist diese Bewegung erlaubt, da ein Labyrinth einen Punkt gemacht hat.

1.4 Wipeoutregel

Neben der Fortschrittsregel gibt es noch die Wipeoutregel: Pro Aufgabe darf pro Weg nur max. 1 Labyrinth über eine Grube wieder zu dem Startpunkt zurückgesetzt werden. D.h. wenn in den zwei parallelen Labyrinth bei beiden mind. eine Grube betreten wurde, so wird dieser Weg als Zeitverschwendung anerkannt und verworfen. Bewegt sich ein Charakter aus einem Labyrinth in eine Grube, darf der Charakter im anderen Labyrinth in keine weitere Grube mehr laufen. Der Charakter, der zuvor schon bereits eine Grube verwendet hatte, darf die Gruben weiterhin verwenden.

Es soll ebenfalls eine Warteschlange erstellt werden, die diesmal alle vier Koordinaten, die vergangenen Bewegungen und gleichzeitig auch die Punktzahl von beiden Labyrinth speichert. Sobald beide am Ziel angekommen sind, wird der Weg zurückgegeben.

1.5 Zusatzherausforderung: Funktion verallgemeinern

Dieses Problem lässt sich ebenfalls bei mehreren parallel verlaufenden Labyrinth lösen. Es soll eine weitere Version erstellt werden, die bei mehreren Labyrinth ein $2 \cdot k$ -Dimensionales Labyrinth erstellt, wobei k für die Anzahl der Labyrinth steht. Dies soll möglich gemacht werden, indem man die einzelnen Operationen nicht mehr ausschließlich auf die zwei Labyrinth bezieht, sondern auf eine Gruppe von

Labyrinthen. Es soll eine Möglichkeit geben, mehrere Labyrinth - die durch eine zuvor vorgegebene Zahl bestimmt wird - zu laden und wie bei einem 4-dimensionalen Labyrinth zunächst einmal alle Labyrinth einzeln zu lösen. Anschließend soll ein kurzer Weg für das $2 \cdot k$ -dimensionale Labyrinth generiert werden. Bei der Fortschrittsregel gilt, dass mind. eines der k Labyrinth Fortschritte machen muss. Bei der Wipeoutregel gilt, dass mind. ein Charakter der k Labyrinth nicht in eine Grube fallen darf. (D.h. Es dürfen maximal $k - 1$ Charaktere in eine Grube fallen. Falls alle Charaktere mind. einmal in eine Grube fallen, wird dieser Weg verworfen.) Die Gesamtpunktzahl besteht dabei aus der Summe der Punktzahlen aller Labyrinth. Dadurch sollen Wege belohnt werden, deren Charaktere gleichzeitig in die gleiche, richtige Richtung gehen (es gibt dadurch maximal k Punkte als Belohnung pro Bewegung). Sobald ein kurzer Weg zum Ziel für alle Charaktere gefunden wurde, werden die Bewegungen zurückgegeben. Da diese Idee nach der Fertigstellung des Dokuments aufgetaucht ist, werde ich diese Idee in einer separaten Datei (Main_n-Labyrinth.py) umsetzen.

2 Umsetzung

Für die Umsetzung verwende ich Python 3 und teile es in 4 Funktionen auf:

- `read_labyrinth` (Umwandlung von rohem Text zu dem Labyrinth, Argument: Text, der die vorgegebene Beispielaufgabenstruktur besitzt, gibt das Labyrinth als Dictionary zurück),
- `solve_labyrinth` (Funktion für das Lösen eines einzelnen Labyrinths, Argument: Feldgröße, vertikale sowie horizontale Wände und alle Gruben, gibt eine Wegbeschreibung als String sowie Koordinatenliste zurück),
- `solve_duo_labyrinth` (Funktion für das Lösen von zwei parallelen Labyrinthen, Argument: Feldgröße, beide Labyrinth als vom `read_labyrinth` zurückgegebenes Datenformat und die Lösung für beide Labyrinth, die in `solve_labyrinth` generiert wird als Koordinatenliste, die Pufferpunkte als Integer und gibt die Wegbeschreibung zurück) und
- Hauptfunktion `main` (behandelt die Interaktionen zwischen allen drei Funktionen. Argument: Dateiname, kein Rückgabewert).

2.1 main

Main ist unser Einstiegspunkt für dieses Programm. Es liest zunächst einmal die Daten von den Beispielaufgaben als String ab (content) und gibt die Informationen an die Funktion `read_labyrinth` weiter, um dieses Labyrinth zu erstellen. Anschließend werden beide Labyrinth zunächst einmal einzeln in `solve_labyrinth` gelöst und main gibt hierbei die erforderlichen Argumente weiter. Beide Rückgabewerte speichert es ab. Die vergangenen Koordinatenliste werden unter `paths` gespeichert und die benötigten Bewegungen unter `movement`. Sobald beide Rückgabewerte erhalten wurden, wird geschaut, wie viele Labyrinth sich lösen lassen. Ist nur ein Labyrinth zu lösen, so informiert dieses Programm den Nutzer, gibt die Wegbeschreibung für das lösbare Labyrinth und beendet das Programm. Bei keinen lösbaren Labyrinthen werden die Nutzer informiert und das Programm beendet sich. Sind beide Labyrinth lösbar, so werden beide Labyrinth in `solve_duo_labyrinth` eingesetzt. Dabei werden die Lösungen bzw. die Listen der Koordinaten zur Lösung aus beiden Labyrinthen weitergegeben und die Lösung wird anschließend angezeigt und das Programm beendet.

2.2 Read_labyrinth

`Read_labyrinth` wandelt anschließend den String zu einem Labyrinth um. Dazu werden die Zeilen zunächst einmal bei neuen Zeilen (n) getrennt und eine Variable erstellt, die den Index der Zeile darstellt. Der Index erhöht sich mit jeder dazu gelesenen Zeile. In der ersten Zeile werden die Größen abgelesen und in Integer umgewandelt. Anschließend werden die horizontalen sowie vertikalen Wände mithilfe der Kombination aus einem for-loop sowie `map` zu zwei-dimensionale Integer Listen umgewandelt. Die erste Dimension steht dabei für den y-Wert (Höhe), während die zweite Dimension für den x-Wert steht. Anschließend wird die Anzahl der Gruben abgelesen und die Gruben werden alle in einer Hashmap gespeichert. Eine Grube erhält als Datenformat ein Tuple mit zwei Einträgen: die x-Koordinate sowie die y-Koordinate. Hashmaps funktionieren mit Gruben am besten, da man nur schaut, ob eine Koordinate in der Grubensammlung enthalten ist und bei solchen Aufgaben Hashmaps eine durchschnittliche

Zeitkomplexität von $O(1)$ besitzen. Die genannten Schritte werden anschließend für das zweite Labyrinth wiederholt und zum Schluss als Dictionary zurückgegeben, welches die horizontalen sowie vertikalen Wände (beide 2-dimensionale Listen) und alle Gruben (als Set) von beiden Labyrinthen enthält. Hinzu wird ebenfalls die Größe der Labyrinth im Dictionary zurückgegeben.

2.3 Solve_labyrinth

In `Solve_labyrinth` werden einzelne Labyrinth mit dem Breadth First Search Algorithmus auf den kürzesten Weg zum Ziel analysiert. Hierbei wird mit den Koordinaten x und y gearbeitet, wobei $x = 0$ sowie $y = 0$ der Startpunkt ist. Besuchte Koordinaten werden in einem Set (visited) gespeichert, um schnell prüfen zu können, ob ein Punkt bereits besucht wurde. Visited nimmt erst einmal alle Gruben (pits) aus der Liste auf, um so Bewegungen in die Gruben zu vermeiden (da Gruben die Charaktere zu dem Startpunkt zurücksetzen, kann es unmöglich einen Weg über die Grube als kürzesten Weg geben). Die möglichen Bewegungen (directions) werden als eine Liste aus Tuples dargestellt, wobei jede Tuple die möglichen Bewegungen als Koordinaten aus x und y speichert. Alle Koordinaten, die mit der Tuple dargestellt werden, beinhalten an der ersten Position die x -Koordinate und an der zweiten Position die y -Koordinate. Die Tuples werden wie mathematische 2-dimensionale Vektoren behandelt. Anschließend wird noch ein Warteschlangensystem (queue) mithilfe einer Liste erstellt, die ein Tuple mit der aktuellen Koordinate, den dafür benötigten Bewegungen als String sowie dem vorherigen payload, welches zuvor verwendet wurde, speichert. Es werden die vorherigen payloads gespeichert, damit man später die Bewegungen mithilfe eines teilweise rekursiven Algorithmus wiederherstellen kann. Dabei kann payload als ein Knoten gesehen werden, dessen Vorgänger zur vorherigen Position zeigt.

Sobald alles initialisiert wurde, werden die Breadth First Search Befehle so lange wiederholt, bis die Warteschlange leer ist oder ein Weg gefunden wurde. Falls die Warteschlange leer sein sollte, gibt es keinen Weg zum Ziel und es gibt statt der Koordinatenliste sowie Wegbeschreibung ein Tuple aus zwei None zurück, da diese Funktion normalerweise zwei Werte zurückgibt.

Zunächst einmal werden die Koordinaten (x, y) von der ersten Position der Warteschlange gepoppt und mithilfe einer for-Schleife nach allen vier Bewegungsrichtungen getestet (Bewegungsrichtungen werden als dx und dy gespeichert). Dabei werden die neuen Koordinaten (nx und ny) durch die Addition der aktuellen Position und der geplanten Bewegungskordinate bzw. des Vektors berechnet. Anschließend wird kontrolliert, ob sich die neue Koordinate innerhalb des Spielbereichs befindet ($0 \leq nx < n$ sowie $0 \leq ny < m$) und ob diese Koordinate schon bereits besucht wurde ((nx, ny) in visited). Abhängig von der Bewegung wird anschließend geschaut, ob sich auf diesem Weg eine Wand befindet.

Bei einer Bewegung nach...

- ... unten ($dy = 1$) wird geschaut, dass `horizontal_walls[y][x]` keine Wand hat.
- ... oben ($dy = -1$) wird geschaut, dass `horizontal_walls[y - 1][x]` keine Wand hat.
- ... rechts ($dx = 1$) wird geschaut, dass `vertical_walls[y][x]` keine Wand hat.
- ... links ($dx = -1$) wird geschaut, dass `vertical_walls[y][x - 1]` keine Wand hat.

Falls eine der drei Bedingungen nicht erfüllt ist, wird die Funktion in eine andere Richtung neu probieren bzw. eine neue Position aus der Warteschlange poppen. Falls alle Bedingungen erfüllt waren, wird der Buchstabe für das Bewegungssymbol gespeichert (movementletter). Für die Buchstaben werden die WASD-Steuerungen für Videospiele übernommen. Dabei gilt: Oben: W; Unten: S; Links: A und Rechts: D. Falls man sich im Ziel befinden sollte (Koordinate: $n - 1, m - 1$), so werden die Koordinaten für die der vergangenen Positionen als eine Liste sowie die Bewegungen als WASD-Muster im String Format zurückgegeben. Dies verläuft teilweise rekursiv, indem man alle Koordinaten sowie Symbole in eine Liste bzw. Kette addiert und so lange wiederholt, bis kein Vorgänger mehr vorhanden ist. Falls man noch nicht am Ziel ist, wird die neue Position als besucht markiert, indem man die Koordinaten in den Hashmap der besuchten Positionen aufnimmt und die neuen Koordinaten für die weiteren Bewegungsmöglichkeiten aus der Position in die Warteschlange hinzufügt, wobei Bewegungssymbol und vorheriger payload ebenfalls im gleichen Dictionary bzw. neuen payload gespeichert werden.

2.4 Solve_duo_labyrinth

Diese Funktion verhält sich ähnlich wie `solve_labyrinth`, jedoch wird diesmal das Ganze (beide Labyrinth) als 4-dimensionales Labyrinth gesehen. Es werden die gleichen Variablen initialisiert wie bei `solve_labyrinth`, jedoch wird diesmal zusätzlich noch die Höchstpunktzahl (integer best) auf 0 gesetzt. Zusätzlich werden Positionen, Punktzahl, Bewegungssymbol, wipeouts sowie der vorgänger Payload in einem Dictionary (payload) gespeichert. Beide Labyrinth bekommen eine eigene Integer Punktzahl (progress1 und progress2) - die bei Folung der richtigen Wegbeschreibung steigt - und eigene Koordinaten (position1 und position2) zugewiesen. Alle Punktzahlen fangen mit 0 an, Koordinaten mit (0, 0) und wipeouts mit einer Tuple aus zweimal False, die je ein Labyrinth darstellt. Visited beginnt diesmal außerdem leer, da die Gruben diesmal die Charaktere tatsächlich zu dem Startpunkt zurücksetzen. Da wir das Ganze als ein 4-dimensionales Labyrinth sehen, werden alle Einträge in Visited eine Tuple aus zwei Koordinaten bzw. vier Richtungskoordinaten (zwei x-Koordinaten und zwei y-Koordinaten) sein.

Sobald alles initialisiert wurde, wird wieder eine Schleife durchgeführt, bis (die Warteschlange leer ist oder) ein Weg gefunden wurde. Dabei wird diesmal der gesamte payload von der ersten Position der Warteschlange gepoppt und wieder in alle vier Richtungen (dx, dy) mithilfe einer for-Schleife überprüft. Die Positionen werden nicht mehr als einzelne x-, y-Werte gespeichert, sondern als ein Tuple aus (x, y). Aus dem payload werden die Positionen als Position 1 (p1) und Position 2 (p2) gespeichert. Neue Positionen (np1 und np2) starten anfangs leer (bzw. als None), um später feststellen zu können, ob sich durch die Bewegungen (Addition von dx, dy) eine Änderung der Positionen ergibt. Abhängig von der Bewegungsrichtung werden anschließend die neuen Positionen berechnet bzw. überprüft, ob diese Bewegung den Spielbereich verlässt oder durch eine Wand blockiert wird. Dies wird differenziert auf die einzelnen Labyrinth durchgeführt. Dabei gilt: Bei einer Bewegung nach... (x: x-Position im Labyrinth, y: y-Position im Labyrinth)

- rechts (dx = 1) gilt: $x + 1 \neq n$ und `vertical_walls[y][x] == 0`
- Links (dx = -1) gilt: $x - 1 \geq 0$ und `vertical_walls[y][x - 1] == 0`
- Unten (dy = 1) gilt: $y + 1 \neq m$ und `horizontal_walls[y][x] == 0`
- Oben (dy = -1) gilt: $y - 1 \geq 0$ und `horizontal_walls[y - 1][x] == 0`

Falls für ein Labyrinth beide Aussagen wahr sind, wird die neue Position durch die Addition der Koordinaten mit der Veränderung ($np = (x + dx, y + dy)$) berechnet und unter np gespeichert.

Anschließend wird überprüft, ob ein Charakter schon am Ziel angekommen ist. Ist dies der Fall, so wird die neue Position durch die vorherige Position ersetzt, da man beim Ziel stehen bleibt. Hat sich eine Position durch die Bewegung nicht verändert ($np == None$), so übernimmt es ihre alte Position wieder.

Nachdem alle neuen Positionensvariablen einen Wert zugewiesen bekommen haben, wird überprüft, ob sich eine Position auf einer Grube befindet. Falls das der Fall ist, wird die Position auf (0, 0) zurückgesetzt und die wipeout Variable (NewWipeouts) wird für das Labyrinth auf wahr gestellt. Dies signalisiert, dass der Charakter in dem bestimmten Labyrinth mind. einmal in eine Grube gefallen ist. Ist der Charakter in dem anderen Labyrinth ebenfalls schon bereits mind. einmal in eine gefallen, wird dieser Weg verworfen, da beide Charaktere mind. einmal in eine Grube gefallen sind.

Anschließend wird überprüft, ob diese Stellung unter Beachtung beider Labyrinth schon zuvor erreicht wurde, indem man überprüft, ob das Tuple aus Position1 und Position2 schon bereits in visited enthalten ist. Falls dies der Fall ist, wird dieser Weg verworfen und ein neuer Weg wird probiert.

Ähnlich wie bei `solve_labyrinth` wird ebenfalls das Bewegungszeichen bestimmt. Falls beide Charaktere das Ziel erreicht haben, werden alle benötigten Bewegungen durch Addition aller vorherigen Bewegungen mit der aktuellen Bewegung bestimmt (alte Bewegungen kommen vor neuen Bewegungen). Dies verläuft teilweise rekursiv, indem man die Bewegungssymbole aller vorherigen payloads zusammenaddiert.

Die Punktzahlvergabe für das Folgen der richtigen Strecke wird ebenfalls berechnet. Dabei wird geschaut, ob die Position die nächste richtige Position ist (`solution[progress + 1] == position -> +1 Punkt`) oder die vorherige richtige Position (`solution[progress - 1] == position -> -1 Punkt`). Falls keiner der beiden

Fälle zutrifft, wird die vorherige Punktzahl übernommen bzw. falls man in eine Grube gefallen ist, alle Punkte entzogen.

Fortschrittsregel: Mithilfe der Punktzahl wird geschaut, ob mind. ein Labyrinth Fortschritt erzielt hat. Falls dies nicht der Fall ist, wird dieser Weg verworfen. Anschließend wird geprüft, ob man zu viele Punkte hinter dem besten (für die zu diesem Zeitpunkt gegebenen Bewegungen) Rekord liegt. Dafür habe ich mir die Zahl 10 (progressPuffer) ausgesucht, da 10 zwischen schnellen Lösungen sowie kurzen Strecken ausbalanciert ist. Für kürzere Strecken kann diese Zahl auf Kosten der Laufzeit erhöht werden. (Hinweis: Ich habe dies nachträglich verändert. Diese Funktion nimmt nun die Variable "progressPuffer" auf, welche standardmäßig auf 10 gesetzt ist.)

Fallen Wege unter den genannten progressPuffer Wert, so ist dieser Weg viel zu weit hinter dem zu dem Zeitpunkt gegebenen Bestwert und wird deshalb nicht weiter verfolgt. Zum Schluss werden die Ergebnisse beim Erreichen einer neuen Rekordpunktzahl geprüft und solche Fälle als neue Rekorde gespeichert.

Zum Schluss wird diese Stellung (da es sich hierbei um ein 4-dimensionales Labyrinth handelt, werden die Positionen beider Labyrinth als eine gemeinsame Stellung gesehen) als besucht markiert, und für die weitere Berechnung ein neues payload vorbereitet, welches die neue Punktzahl beider Labyrinth, die Positionen, das vorherige payload und die Wipeouts enthält, um es in die Warteschlange hinzuzufügen. Dieser Schritt wird für alle vier möglichen Bewegungen (Oben, Links, Rechts und Unten) durchgeführt, wodurch immer mehr Möglichkeiten zum Ziel entstehen, aber gleichzeitig auch schlechte Wege automatisch ausscheiden. Falls kein Weg gefunden wurde, wird diese Funktion nochmal ausgeführt aber mit einem höheren progressPuffer, um zuvor ignorierte Möglichkeiten erneut zu prüfen. Dies garantiert einen kurzen Weg zum Ziel bei gleichzeitig kurzer Laufzeit.

2.5 Umsetzung des solve_n-labyrinth

Um die Idee, beliebig viele Labyrinth bei gleichzeitigen Bewegungen auf kürzestem Weg zu berechnen, umzusetzen, müssen die Dateien gezielt verändert werden. Dafür sollen in den Beispieldateien neben der Größe des Labyrinths auch noch die Anzahl der Labyrinth als Zahl angegeben werden. (z.B. 100 100 3 -> n = 100, m = 100, k = 3).

Read_labyrinth:

Diese Funktion nimmt neben n und m die Anzahl der Labyrinth k als Integer auf (da ich erst später die Anzahl der Labyrinth auf k benannt habe, ist in der Quelle die Anzahl der Labyrinth als nLab bezeichnet worden). Mithilfe einer for-Schleife werden abhängig von der Anzahl der Labyrinth mehrere Labyrinth geladen und in einer Liste als Dictionary gespeichert. Es werden deshalb nicht mehr zwei Labyrinth in ein Dictionary gespeichert, sondern pro Labyrinth ein Dictionary Eintrag erstellt. Dazu kommt, dass die Größe der Labyrinth nicht mehr im Dictionary bzw. in der Liste gespeichert wird, sondern separat mit der Labyrinthliste zurückgegeben wird. Durch diese Veränderung ist es möglich, beliebig viele Labyrinth zu laden.

Solve_labyrinth:

Keine Veränderung. Diese Funktion nimmt nur ein Labyrinth auf und benötigt daher keine Veränderungen.

Solve_n_labyrinth:

Diese Funktion kann beliebig viele Labyrinth aufnehmen und dabei den kürzesten Weg finden, unter der Bedingung, dass alle Labyrinth eine Lösung besitzen. Neben Labyrinthgröße (dimensions) und Pufferpunkten (progressPuffer) nimmt es nun die Labyrinth (labyrinths) und die Lösungen (solutions) als Liste auf. Payload nimmt diesmal alles, was vorher nur auf zwei Labyrinth bezogen war, in k-facher Ausführung auf. Dazu zählt unter anderem, dass es k Punkte (progress), k Positionen (positions) und k Wipeouts gibt. Punkte und Positionen werden daher als Liste gespeichert, während wipeout als Tuple gespeichert wird (da wipeout meistens unverändert bleibt und deshalb nicht jedes Mal eine Liste dafür initialisiert werden muss). Die restlichen Sachen werden wie bei solve_duo_labyrinth initialisiert. Ein massiver Unterschied ist, dass hier alle Operationen nun mithilfe einer for-Schleife (welche die Schritte

für k wiederholt) in allen Labyrinthen durchgeführt werden. Es werden in allen Labyrinthen einzeln die neuen Positionen berechnet, dazu zählt u.a. die Bewegung selbst, ob das Ziel erreicht ist und ob sich ein Charakter in eine Grube bewegt hat. Für die Wipeoutregel habe ich außerdem noch die boolean Variable “cancel” erstellt, die bei der Wipeoutregelbruch (also, wenn alle Charaktere mind. einmal in eine Grube gefallen sind) signalisiert, dass dieser Weg verworfen werden soll. Cancel wird bei jeder Bewegung automatisch auf False gestellt, da es von den neuen Positionen abhängig ist. Falls jemand über einer Grube zu dem Startpunkt zurückgesetzt wird, wird zunächst einmal payload geupdated und anschließend geschaut, ob mind. ein Charakter noch nicht in eine Grube gefallen ist (False in newWipeout). Falls bereits alle die Grube verwendet haben, wird cancel auf True gesetzt und die Schleife für das Labyrinth abgebrochen. Durch das Setzen von cancel auf True wird die komplette Bewegung abgebrochen. Koordinaten werden in ein Tuple aus einzelnen Koordinaten umgewandelt und in der Liste der besuchten Stellungen abgeglichen. Mithilfe von “not (False in (i == goal for i in nps))” kann überprüft werden, ob alle im Ziel angekommen sind, um die restlichen Zieloperationen durchzuführen. Punkte werden mithilfe einer for-Schleife für einzelne Labyrinth berechnet und mithilfe der Formel: “True in (payload[,progress][nLab] < newProgressList[nLab] for nLab in range(nLabyrinths))” kann überprüft werden, ob mind. ein Labyrinth Fortschritt erzielen konnte. Für die Berechnung der Gesamtpunktzahl wird die Summe der Listen verwendet und die nächste Payload erhält ebenfalls die ganzen Listen/Tuples bei Punktzahl, Positionen und Wipeouts. Besuchte Stellungen werden als ein gemeinsames Tuple von neuen Positionen gespeichert. Sonst verläuft diese Funktion wie solve_duo_labyrinth.

Main: Hier werden alle Variablen, für die vorher zwei separate Variablen verwendet wurden, durch eine Liste mit dem gleichen Verwendungszweck ersetzt. Hinzu kommt, dass die Labyrinth nach ihrer Lösbarkeit gefiltert werden. In eine neue Liste (newLabyrinths und newPaths) werden lösbare Labyrinth aufgenommen und anschließend an solve_n_labyrinth weitergegeben.

3 Analyse des Programms

3.1 read_labyrinth

Die Komplexität der Funktion ist hierbei direkt abhängig von der Größe des Labyrinths. Wenn man sich die Funktion anschaut, gibt es $(n-1) \cdot m$ vertikale Wände, $n \cdot (m-1)$ horizontale Wände, sowie $g1$ und $g2$ Gruben. Natürlich gibt es auch Operationen, die eine Zeile ablesen (z.B. wenn man den Wert von n , m , $g1$ und $g2$ abliest), da aber diese Operationen $\mathcal{O}(1)$ dauern, kann es aufgrund der Existenz von deutlich größeren Größen ignoriert werden. Die Summe aus den einzelnen Komplexitäten lässt sich zu $\mathcal{O}((n-1) \cdot m + n \cdot (m-1) + g1 + g2)$ bilden, die zu $\mathcal{O}(n \cdot m)$ vereinfacht werden kann (da $g1 + g2 \ll n \cdot m$ ist). Dies ist für die Zeitkomplexität als auch Speicherkomplexität gültig.

3.2 solve_labyrinth

Da wir im solve_labyrinth mit dem Breadth-First Algorithmus arbeiten, gehen wir zu jeder Position maximal einmal hin. Dabei gibt es $n \cdot m$ mögliche Positionen, weshalb es zu $\mathcal{O}(n \cdot m)$ führt. Da ich alle Gruben in der Funktion als ein Set in der Variable visited einspeichere, wobei Positionen auf die Existenz einer Position in der Hashmap getestet werden, ergibt sich $\mathcal{O}(1)$. Schlussendlich lässt sich dies zu $\mathcal{O}(n \cdot m \cdot 1)$ summieren und zu $\mathcal{O}(n \cdot m)$ vereinfachen, was auch der Laufzeitkomplexität entspricht. Im Bereich der Speicherkomplexität wird deutlich, dass einige Variablen von der Größe des Labyrinths abhängig sind. Dazu gehören u.a. visited und queue. Durch die teilweise rekursiv verlaufende Datenstruktur der Warteschlange queue, lässt es sich ebenfalls durch $\mathcal{O}(m \cdot n)$ darstellen. Das liegt daran, dass die Warteschlange nur einzelne Positionen sowie Symbole speichert, wodurch die Speicheranforderungen nicht mit der Zeit wachsen (bezogen auf movement und position) und nur mit mehr Koordinaten (durch das Hinzufügen von Positionen bzw. vorherigen payloads) wächst. Schlussendlich lässt sich die Speicherkomplexität zu $\mathcal{O}(m \cdot n)$ zusammenfassen, da alle weiteren Faktoren maximal a^2 komplex sind, wobei a für alle Variablen steht (bzw. Any Variables).

3.3 Solve_duo_labyrinth

Diese Funktion verhält sich ähnlich wie solve_labyrinth mit dem eindeutigen Unterschied, dass es mit einem 4-dimensionalen Labyrinth arbeitet. Deshalb gibt es insgesamt $n^2 \cdot m^2$ Möglichkeiten die je

maximal 1-mal auftreten. Es lässt sich aus diesem Grund zu $\mathcal{O}(n^2 \cdot m^2)$ umwandeln. Da beide Gruben als Set vertreten sind und regelmäßig Positionen auf deren Existenz im Set getestet werden, ergibt sich $\mathcal{O}(1)$. Schlussendlich lässt sich dies zu $\mathcal{O}(n^2 \cdot m^2 \cdot 1)$ bilden, die wiederum zur Laufzeitkomplexität: $\mathcal{O}(n^2 \cdot m^2)$ zusammengefasst werden kann. Im Bereich der Speicherkomplexität wird deutlich, dass visited mit mehr Positionen immer größer wächst. Dieses Wachstum lässt sich mit $n^2 \cdot m^2$ darstellen (da es x1, x2, y1 und y2 zugeordnet werden). Hinzu lässt sich die Warteschlange queue ebenfalls mit $n^2 \cdot m^2$ zusammenfassen, da es wieder teilweise rekursiv verläuft und die einzelnen payloads nur einzelne Positionen sowie Bewegungssymbole speichert. Schlussendlich beträgt die Speicherkomplexität $\mathcal{O}(m^2 \cdot n^2)$. Weitere Faktoren sind hier wieder höchstens a^4 komplex.

3.4 main

Unser Einstiegspunkt liest zunächst einmal die Inhalte der Beispieldatei, weshalb sich $\mathcal{O}(\text{content})$ ergibt (content steht hierbei für die Größe der Datei). Da der Einstiegspunkt einmal parse_labyrinth, zweimal solve_labyrinth und maximal einmal solve_duo_labyrinth aufruft, lässt sich die Komplexität aus der Summe der Komplexitäten der einzelnen Funktionen bilden. Daraus ergibt sich: $\mathcal{O}(\text{content} + n \cdot m + n \cdot m)$ wenn nur maximal ein Labyrinth lösbar ist oder $\mathcal{O}(\text{content} + n \cdot m + n \cdot m + n^2 \cdot m^2)$ wenn beide Labyrinth lösbar sind. Somit ergibt sich bei Vereinfachung: $\mathcal{O}(n \cdot m)$ für maximal ein lösbares Labyrinth und $\mathcal{O}(n^2 \cdot m^2)$ für zwei lösbare Labyrinth. Dies ist für die Laufzeitkomplexität als auch Speicherkomplexität gültig, da main unter Nichtbeachtung der Funktionen solve_labyrinth, solve_duo_labyrinth und read_labyrinth nur höchstens $\mathcal{O}(\text{content})$ erreichen kann.

3.5 Einstufung des Problems

Dieses Problem ist ein Teil von P. Das liegt daran, dass die Labyrinth durch die Eingrenzung von n und m endlich sind. Wenn man die Einschränkungen minimieren würde (progressPuffer = unendlich), ist es möglich, einen (oder mehrere) absolut kürzesten Weg zu finden, da die Laufzeit auf höchstens $\mathcal{O}(n^2 \cdot m^2)$ beträgt. Ebenso ist dieses Problem Teil von NP, da die erzeugten Lösungen auf ihre Gültigkeit überprüft werden können, indem man die Charaktere selber (bzw. mithilfe eines Programms) steuert und darauf testet, ob beide am Ende ankommen.

3.6 Analyse des verallgemeinerten Labyrinths ($2 \cdot k$ -dimensionales Labyrinth)

Read_labyrinth: Im read_labyrinth kommt durch diese Änderung der Faktor k noch dazu, wobei k für die Anzahl der Labyrinth steht. Da Gruben und Größen deshalb nun von k abhängig sind, ist die neue Berechnung: $\mathcal{O}(k \cdot (n \cdot m) + g_1 + g_2 + \dots + g_k)$. (g_k steht hierbei für die Grube mit dem Index k). Dies lässt sich zu $\mathcal{O}(k \cdot n \cdot m)$ zusammenfassen und gilt für die Laufzeitkomplexität als auch Speicherkomplexität.

Solve_labyrinth: Im solve_labyrinth verändert sich durch die Veränderung nichts, da es weiterhin nur ein Labyrinth aufnimmt und deshalb von k unabhängig ist.

Solve_n_labyrinth: In solve_n_labyrinth nimmt die Laufzeit- sowie Speicherkomplexität durch diese Veränderung mit größerem k massiv zu. Da wir hier mit $2 \cdot k$ -dimensionalen Labyrinth arbeiten, gibt es daher auch $(n \cdot m)^k$ Möglichkeiten. Aus diesem Grund beträgt die Laufzeitkomplexität $\mathcal{O}(n^k \cdot m^k)$ bzw. $\mathcal{O}((n \cdot m)^k)$. Hinzu kommen die einzelnen Möglichkeiten, die unter visited alle gespeichert werden, weshalb die Speicherkomplexität ebenfalls $\mathcal{O}(n^k \cdot m^k)$ beträgt.

main: Bei main übernimmt es die Komplexität von solve_n_labyrinth, da main diese Funktion direkt aufruft und main selber eine kleinere Komplexität gegenüber solve_n_labyrinth besitzt, weshalb man es ignorieren kann. -> Speicher- und Laufzeitkomplexität: $\mathcal{O}(n^k \cdot m^k)$

4 Einstufung des Problems:

Im Falle von $2 \cdot k$ -dimensionalen Labyrinth befindet sich dieses Problem weiterhin in P. Das liegt daran, dass es einen natürlichen (ohne 0) k-Wert geben muss, da sonst diese Funktion nicht funktionieren würde. Es ist (meiner Meinung nach) nicht möglich, den Wert von k mithilfe einer mathematischen Funktion oder sonstiger Formeln darzustellen, wenn k der Index eines Labyrinths ist und die Labyrinth von der

Aufgabenstellung gegeben werden müssen. Aus diesem Grund kann k maximal nur so groß sein, wie die Anzahl der Labyrinth. K besitzt daher die Eigenschaften einer konstanten natürlichen Zahl (ohne 0), die durch die Anzahl der Labyrinth begrenzt wird.

Sobald ein k -Wert gegeben wurde, lassen sich die Laufzeit- sowie Speicherkomplexitäten als ein Polynom darstellen. Eine Einstufung als NP ohne P oder NP-Hard wäre erst gültig, wenn man es irgendwie schaffen sollte, einen Generator für die Labyrinth zu erstellen, dessen Größe sich nicht mithilfe einer Zahl darstellen lässt. Dies kann ich mir jedoch nicht vorstellen und es ist bei unseren Labyrinth nicht der Fall, da unsere Labyrinth eindeutig durch n und m eingeschränkt sind.

In der Theorie wäre es deshalb möglich, mithilfe von k ein $2 \cdot k$ -dimensionales Labyrinth nach dem kürzesten Weg zu berechnen, jedoch würde es in der Praxis bei einem großen k -Wert (sowie angemessenen n - und m -Werten) sehr lange dauern (bzw. zu lange dauern), bis es berechnet ist.

5 Beispiele

Die Bewegungen werden wie folgt dargestellt:

W	Bewegung nach oben
A	Bewegung nach links
S	Bewegung nach unten
D	Bewegung nach rechts

filename: labyrinth0.txt

Die Aufgabe aus dem Arbeitsblatt, mit der bestmöglichen Lösung unter 1ms.

Amount of Actions: 8; Required time: 0.0006365776062011719

SSDWWDSS

filename: labyrinth1.txt

Hier wieder mit der bestmöglichen Lösung. Bereits hier zeigt es, dass es funktioniert.

Amount of Actions: 31; Required time: 0.0013513565063476562

DDDDSSAWASAWASSDWSDWAAWDDDDSSS

filename: labyrinth2.txt

Gruben wurden richtig behandelt. Dies erkennt man daran, dass der Charakter aus dem zweiten Labyrinth zunächst einmal in eine Grube bei (1, 1) geht und anschließend durch zwei Bewegungen nach unten in die richtige Richtung geht.

Amount of Actions: 65; Required time: 0.009835004806518555

DDSSDDDDWDDDSASDSDSASAWAAAWAASSASDSDDDDDSSDDWWDDWDDDDSSA-SASSDDS

filename: labyrinth3.txt

In diesem Labyrinth lässt sich erkennen, dass dieses Programm auch funktioniert, wenn n und m nicht die gleichen Werte besitzen. Hier wurde wieder die bestmögliche Lösung unter 1 Sekunde berechnet.

Amount of Actions: 166; Required time: 0.05328536033630371

SSDWSDDDDDASAAASASSSSSDSDWDDSSSSASDSAASSSSSAWAASASDDDDSSASSDDSS-DSSDSASASSSDWSSSASAWAWASSSSSDSASDDDDSSDDSSASAAAWASSSASASSD-SAASAASASDWSDSDSDWDDDDWDDWSSASSSD

filename: labyrinth4.txt

In Labyrinth 4 wird es deutlich, welche zeitlichen Vorteile dieses Punktesystem hat. Bei Entfernung des Punktesystems habe ich für dieses Labyrinth mehrere Stunden gebraucht, um es in Google Colab zu berechnen, wobei ich nicht weiß, ob es tatsächlich geschafft wurde (ich habe es nur noch mit großen Zeitabständen angeschaut und es irgendwann vergessen). Da Labyrinth 4 keine 3-Ways oder 4-Ways besitzt, ist es kurz zusammengefasst ein langer Weg, der am Ziel endet. Deshalb kann mit jeder möglichen Bewegung maximal 2 neue Möglichkeiten entstehen, die je ebenfalls 2 neue Bewegungen verfügen. Deshalb ist in diesem Labyrinth dieses Punktesystem sehr wichtig, da vor allem in dieser Aufgabe Wege belohnt werden sollen, deren Charaktere beide gleichzeitig in die richtige Richtung laufen. Wege, die zu viele Möglichkeiten zu gleichzeitigen Bewegungen verpassen, werden systematisch verworfen und dadurch ausgefiltert.

Hinzu kommt noch, dass für ProgressPuffer = 13 die nächstgelegene Best-Lösung (14446 Aktionen) in 8.307620286941528s berechnet wird. Weitere Informationen sind in der Datei „labyrinth4_besteausgabe.txt“ zu finden.

Amount of Actions: 14561; Required time: 4.317425966262817

DDDSSAWAASDSASDSASDDSDWWAWWDSWDSSASSASAAWASSSDWDDSSSDWWDD-
 WAAWDDDDDDWW-
 SAAWWWWASASDSASDDDDSAAAWDDWWDWAASAWWDDDDSSSSDWDDDDDDDS-
 SAAAAWAWWASSASDDSDWAAWDDDDSSDWSSSAASAASDDDDWAAWWDSSASS-
 SDWDDWAAASAWDWDDDDSSSSDWDDWDDWAAASAWWDDDDDDSSASSASSAWASA-
 ASSDWDSSSSSWDSWDWAWDDDDWAAAWDSWDWDDDDSAASDDSAWASDSASDDD-
 SAASAAWWASSAWDWWASSASAASDSAAAAWDDDWAAAAWAWWWWASSASDSAS-
 DSAASDDDDWWDSSSSDSASDDSSDDDDWDSDDSAASAASAWAAAWDDWASAWWWWASS-
 SSSSAWWWWASSAWWWASSSDSAAWDWAASWWWWWWDSSASSAASDWWDSSDW-
 WAAAWDWWWDSSDDDDWWDSSWASASSAAASAWWWWDSWDWDDWAAWWWASSS-
 SAWWWWAAASAWWDDDDDDWAAAWASAWASAASSSSDDDDWSSSAWASAWSSDDDDSA-
 ASSAWWASSSSSSSSSDSASSSDWWDWDDSDWWDWWAAAADDDWDDWAWDDSSSS-
 SAWWASAAAASSSDWDDSDWWDSSSSDDWAWDWASAWWDWDDDDDSAAA-
 SASDWDDDDWDDSDASDDDDWAWDWWWDSDWDSWDSSDWWDSSSSAWASSSSAAWDWAS-
 SAAWDWAWWASDDSSSAASDDDDDDDDSSAASAWWASAWA-
 SAWASDDDDSAAAAASDWDDSDWDDWDDWWDWDSAAASDDSSSDWAWWDSSDDWW-
 SAAAWASAAAWDDWAAWAAASDDSAASSSAASDSAAWWWWDSWDWAWDWAASAW-
 WAAWWWWWDSSSDWWDSSD-
 WAAWAAAWWWWAAWDDDDASDSAASSSSSDWDDDDWWDWSDWDSAAASDWSSDDDD-
 SAAASDDDDSSSAASDSWAWDWAWDDSDSDWDSAAASAWWAAWWWWWASSDDSSS-
 SAWWWASSSSSSDDSDWAAWDDDDSSDDW-
 SAWWWASSAAAWASAWWASSSSSSDDDAWDWASAWDDWDSWDSSDDWAWWWDDDDSD-
 WAAWWDSDWDDDDDDDDSSSDWWDSSASSSAAAASDDSDASDSASDSSAWAWASDSD-
 SAAAWWWDDWASAAAASSSSSSSDWWDWAAAAWDDSSDDDDWDSWWDWDD-
 WAAASAWWAWDDSDDDSSSWASSAAASDSASASDDWDSAAAAAWDWWWDWAA-
 SAAAAAWASDSDDDWDDSSSSSDWDDWAWDDDDSAASDSAAAASSSDSASSSDWWDSS-
 SDWDDWAAWAWDDSSDWWDWDDDDSSASDSAAASDDSSSSSAWWWWASSDDSS-
 SAASDDSDWDDDDSAWDWWAAAWWDSSDWDDWDSDDWAWASAAASAWDWD-
 WAAWWWWWDSDWASAAASSAWDDDDSAASDWDDDDDDDDSDASDDSSSDWDS-
 SAWASAWWAWWASSSADSAAWDWAWAWDWDWAS-
 SAWDDWAASDSAWAWWAASDSASDWWDSDSAASDDDDDDSAASAWWSDDDWAWDDSDWAW-
 WASSSDSAWWDSDWWDSDSDSAWSSDDDDWWDWAAWDDDDSSSDWDDDSAAASDWSSSSS-
 SSSAWASDSDSDWWDSSDDDDSAAAWASSSDWWDSDSAASSAASAADWAWWDSSDW-
 WAWDWWWWWWDSDWWDWASSAWASSAWDWAADDDWASAASSSSAASDDDDSA-
 ASSAASSSSSDWDDWAAWWDSDSASDDWWDSDSAASDSASSAASDDDDSAWAWAAAWD-
 WAASDSAAASDDSSSDWWDWDSWDSSAASDDDDWAAWWDSDWWDSSASDSS-
 DSSSAWWWWASSDWSDSDSSSAWASSADSAASAASAWWDSDWASAWWDDWAAA-

SASAWWWASAASDDSAASSDWDSSAAAWWASSAWWWDWAWAAWDDDS-
DSSDDWAWDWDDWAAASAWWDWASSAAAWDDWASSSSSSAASAWAASDSSSASDDWWWDSS-
SDDSSAWASAWAWWASDSDDSSASDWWDSSSSAWASSSDWSDWDSAAASDDDWDSDDWA-
WAAASAWWDWAASSAWWWDWAWDWDSDDWDSASDSDDWAWDWWWASSAWWA-
SAWWDDDDWWDSSSAASDSSDWWDWWDSSDWWDSDWSDWAWAWDDDDSSSSSSAW-
WASSDDSAAWWAWDWASASAWWASSASDDDDSSDSSDDWWDSSASDWWDSSAWDWAA-
WASASAWWDWWDSDWWDSDDDWAWASSAWASAWDDDDWDDWAAAWDDDDWAAAAA-
WASDSDDSAAWWAWWAWWDSSSDWAWWWWASSDSASAWWASAWASSDDSDDDWDSASS-
ASSAASSSAWWAASDSAAAAAASAWWWASSAWDWASSAWDDWAWASAWWDWASAWWA-
SAWWWDSDDWASAASDDAWDDWDSAAASDSDWDWAWDDDWWDSSAWASASDDWWDSSDS-
DSAAWWASSSAWAWDWAWASSAWWSDWDSASSSSSDDDDSSDS-
SSSDWWDWWDSDDDWDWAASAAWSAAWDWDDWDSDWAAASSSSAWWASSAAWAWDD-
WAAAWDDWWDSSDWWDSSASDDSSSDWAWDDDDWAAAWASSAWAWWAAAWDWAWD-
SASDWWD-
SAWAAASDWDSAAAWASSDDDDWWDWWDSDWDDDDSSAAAAASDWWDSDDDDDSS-
SSSAWDWAAWWASAWWASSDWDSAAAWWDSDWWDWASSAWWWWDSSDWWDSSSS-
SSDWAAWSAWWWAAAWDSSSDWWDWAAWWDSDWDDWDSSSSSAASDWDDWDDWWD-
WASSAWDDWWDWAAWASSAWWWDSDSDWAAAWWWWWAWDDDDWDS-
SSSDWWDWWDSSDDSSSSAAAWASSSASSASDDWWDWWDSSDDSSWASAWDDDDDDW-
WASSSAASAAWWDSDASDSWSDWWDSDSSSSSDWWDWAWDWAWWWDDSDASDSSSDSSA-
WASSASDSWWDSDWWDWWDSSDWAAAWWWDWAWDSSDDSDSDWDSAAASDDDDSDDD-
SAASAWAAWASSAWWWWWAAWASSSSAWASAASDWDSWDDWWDSSSSSSASDSA-
WAWDDWASAASSAWWASAWASDSSDWDSAAASDDSSDDWAWDDSDSD-
SAASAWAAWAAASAAASDDSAASAWSDWDDWWDSSSSAWASAWWDDWASAWWWAS-
SAWDWWDSDDDDDSAASDDWWDWAAWAAASDSAAWDWWDWSDDDWAAAWWDWDS-
DSSDDSSAAASDSASDDDDDDSSSDWWDWWDWASAADDDWAWDWWDWDDSD-
SAAASDSAAAWDWAAASDDSSSDDDAWDWAAAWDDWWDWWDSSAASDDDWASAWAA-
WASSDDSSAWASSAWASSSSSDWWDSDDDSDSDDWASAAAWDDWDDWDSASDDWWDW-
WAAWDWDSDDWWDSDWAWDWAAASAWWDSDWAAAWWASAWDWAAAAAASSSSASA-
WAWWDWWDSSDDWAWDWAAWDDWWDWASAASDDSSAWASSAWWASSDDSSDSASSA-
ASSAS-
DSAAWWDWWDWAAASDSASAWASSAWWWWWWWWDWASSSSSSAAAWWDSDWAAAD-
SAASSASDDDSAAAASDDDDWAWWWDSDSSAAASDDDDWAWDSDDWAAWDDWWDW-
SAWASASSASSAAWAWDDWAAAWDDDDWWDSSSSDWWDSDWAWDSDDWDSDDWAA-
SAWASAWAAWDSWAWDSSWASAWWASDSSDWWDSDSSDDSDSA-
WASSAAASDDWWDWDSDSAAWWASSAASDDDDSDWWDSDSASSSSSSASDDDDSS-
SSDWAAWWDWSSWASAAAAASDDDDSAASAAWDDWWDWAAAWWDSDWWDW-
SAWAWDDWDSDDSDSAWAAASDDSSDDDDWWDWASAWWAWWDSSDDSSSDW-
SAWASAWWSDDDWDSAAASDDWWDWWDWAAAWDWAAASAWDDDDWDSDDSSDDSA-
WAASDDSDSSAAAWDDWAWASAWWAAWASAAAWASDSSSDWWDSSSAWASS-
DSASDDWAWDWDSAAASDDSAWASDDSAWDWWDSSDWWDWDDWSAWAASDDSSA-
WAWDWASDDDDWAWDDDSASDDWWDSDWWDW-
SAWASAAWDWWDWASSASDSDSAAWAAASDDSAASAWWWDDWAWWDSDDDWAW-
WASSAWWAWDDWAAASAWAWWASDSDSAWASDSAAWWASSAWDWAWDWAAWASSAW-
WASSAWWDSSDWSDWWDSDDDWWDSSSDSASSDWAWDSSAWWWASSSSSDSASDDSS-
SAAAWDDWAAWASSAAWDDWDDWAAWDDSDSSDDWAWDWAAAAASDDSSAWWW-
WASDDDDSSAAASDSASDWWDSSSDSDSDWAWWWAWDSDDSDASDDSAASAWASSDWDS-
SAAAWASAWWWDSDW-
WAAWAWDWAAASSASDWSSSAWWDWAAASSSSDWAWWWWDSSDWAWDSSSAWASS-
ASSAAWDWASSAWWASSAWWDDWWDWWDSSSAASSAWAAASAWWDDWDSWDDW-
WASSAAASAWWDDWAWWDSDSDWWDWSDWWDWWDSDDDDDWWDWWDSSSAW-
WAAWAAWDSDDWAAWWDWASAWWDDWAAWWDWAAASAAAAWASSSSSSAASDDSSA-
WAAWWDWWDSSSDWAWWWWAASDSSAWWDDDDASDSAASDWSSSSSSASDD-

SAASAWAAWWASSSAWWASSSSSSDWWDSSSSAWWWASAWAWDDWAAASAWA-
SAWDDDDWWASAWWWAWDWASAWWASSSAAAWASASDDDSAASDDDDWAWDDSS-
DSSDSSSAAWDWAWWAAASDDSAASDWWDSSAASDDWWWWDSSSDDSDSSSAWWASAWAA-
WASSDWDSSSAWAAWASSSDWWDSDSSSSSAWAAWWASSSSAAAWDDWAAAWDD-
WASSSAWASSAWDDWAAWAAWDDWWASAWDDASDSDDSAASSSDWWDSDSSAWDWA-
SAWWSDDWDSWDSSAASDDSAASSDWDSAAADWAAWAWDWAAWWAAWWDDWAA-
ASSSSAWWAWWASAWDDDDWWWASSDDDSAAWASAWASSDDSSAAAWWWWASSSS-
SAWDDWAWWDDWWDWAWDWASSSWDDDSAWAASDSASDDWWSAAAWDWAWA-
SAWWASAASSAAWWWAAASSSSDWAWWWWDSSSDDDDDWWSAAWASDSAWASDDD-
SAAAAWWDDWDDWAAASSSSSSSSWDSDWDSSAWASSDSASDDDDSAASSDWDSAAASSSSSS-
SSSDWDSWAAWDDDDSSSDWWDSDASDWWWWASDSAWWASAWWDWAWDWAWWASSS-
DSAWWWWAWDWAWWDSWDAWAAAWASSSSDWWDSSSSDDSDSSAWWAWWWAAWASSSS-
SSSSSSSAASSDDWWSAAWAAAWWWWDSSSDWDSASDSASDSDWDAWDDDDWSDW-
WASSAWWWWASSAWWWDWAWDDWAAWWDSDASDDSSASDSSDWDDSSASSDW-
WASAAAWWDSSSASSSSASDWDDDDSDWAWWDDWWWWWAWDWAWWASSDSAAWD-
WAAWDDDDWDDWASAWWASSAAAWDWAWWWDWAAAWWDSWDDWWWWWDW-
WAAWDSDDWAAWDDDDWAAWDDWWWWASDSSSAWASAWWWWASSAWASSASDSDW-
WAAWDDWWWDSSSDWWDSSSDSAASDDSAASAAAASDDDDWWSAWWASDDSSAWAS-
SAWWW-
WASAWAAWWSDDSDWAAAWWDDWDDSSSAAAWASSASDDWDDWWDSDAAA-
ASSDWDDDSAAASAASSAASDSSDWWDSSDWDSWAAWWDSDDDSDSAAWASASDDDSA-
ASSAAWAAAAAWAAWWWASSAASDWWDSDAASDWDSAASSSDSASASSSDWWDSSSSA-
WASSDWWDSSDSAWWASSSSAAAWSDDDWWDSSDDDDWAWWAAWDDASSSSSSDSASSSS-
SDDDWAAAWWWWDSSDWWAAWDDDDWWWDSDWWDSSSSSAWASSSAASDDDDSAAS-
SAWASSDDSSDWAAWDSDDDDDDDDDDDDWDSWAAAAAAAWWWASSAWAAWDD-
WAAWDDWDSWAAAWDSSSDWWDSSSDWWDSSASDDWDDWAAWAAAAAWDSS-
SDSDWWDWASSAWWWWDSSSDWWDSSDDWAWWWDDWDDWASSSAAAWDWWDSD-
SAADWWWAAAAASSDWDSSSAAAWASASDSASDDSAASAAAASDDSSDWWDSSSDSSAWA-
WASAWWAWASDSSAWAWWASAAASDDSAAWASSAWWASSDDDDDDDDAWWWWDSDW-
WAWDDSSDWWDSDWAAAWWAWDWASAAWDDWWDSDWDSDDDDDDSDASDDWWWAS-
SASAAAAWDDWDWAAASDDSAWASDSASDSSDWWDWDDDDDDWDSDDWDSWAWWDSS-
SDWDDWDDDSASAWASASDDDDWDSDDDDDDDDSDWDSWDSSSSDWWDWWAAW-
WAAWWDSDWDDDDSAASDDDDDDSSSDWWDSDSDWAWAAWDWAAASSAWWDDDDDSAS-
DSDDWDSSSDWDSWAWAAWDDDDWAAASAWWDDDDASDSSDWWDSSSDSAASDSAAAS-
DSSSDSAAWDWAAWWWAWDDDWAWASAAS-
DSSDDDSAAASDDSSAAWASAWDWAAAAASAWWWASSSSAADWAAWDSDDDDWWWDSSASS-
SDDSAASSDWDDWDSWDAWAAASAWWAAWDDWAWDDSSAASDWDSWWDWWSAAW-
SAWWASSSSAWWASSAWWWDWAAWASSSSAWWASSAWAWDSWWDWWDWWDWASS-
SAWWWASSSAWWAAAWDSASDWDDSAASDDSSAWDWWWAAAWAWDWDSDDWAWAWD-
WAWDWAAAWDSSDWWDSDSDWAWDDWWWDWDDWAAAWDDWAAASSAASDSAA-
SAWWDWASAWASDDSAASDDSSSAAWDWASAAAWWWASAWWASSAWWAAAWWASS-
SDDSAAASDDSSDWWDWSSDSASDDWWDSSAWASDSSSAWWWASSAWAAAWWWWASS-
SSAWWWASSAWASAWWDDWAAASDSSAWASAAASDDSSSAASDDSSDWWDSDSDWDS-
SAASDSAWWWWASDSSSAWASAAAWWASSSSDDWWSAAWASAAWSDWDDWDSSSSAAA-
WASSSSAWASDWAWDWAAAWDDSSDWWDSDWWWASSSAWWDWAWDDWAAAWDSASS-
SASDDWWDSSSSSAAAASDDDDSDWDSASDWWDSSAASDDWWWASSAWDWAWDWAA-
SAAS-
DSASDSAAWASDDDDWAAWDSSSSSSDWAAWWWWWWWDWAWWWASSSAWWWWWDSD-
DSDSDSSAWASASDSSDWDDSAASDDSSAWASSSDWDSWAAWWDSSDWDSSSDWDSASS-
SSASDSDSAASDSASSAASDSAAASDWDDWAAWDDWWDWWDWAAASAAWAWWAWDD-
WAAWWDSDWDDSSASDSASDDWWDSSDWWSAAAWWAAAWDDDDWAWWASSA-
SAAWDDWASSWDDSDWWDSSDWDAWDDDDDDSSDWDDWWSAWW-
WAWDDDDDSAAASDDDDSAASDDWDDWWSAAWASAWWWASSSAWAWDWAAASSAWASAW-

WASSWDDSDWDSDDDWAAAWDDWDDASDSASAASSSDSASAWWWWWAAWDDSDDDWA-
WAASAWASASSSSAWWWDSASDDDWAAWDWASAWDWDWWDAAWVWAAWWWWDSSDD-
SAADWAASDDSSDDSAASAWWASAWDWDWDDWAAAAWDWAWDASDSASDDSSASDSSA-
WASSDDSSSDSSSAASDDASDWDSDDWWAWDWDWAASAWWWDSWDWAAWWDSDDSS-
DSASSAWASSDSSDSAAASDSAAWASSASDWWDWWDWDSDDDDDDWWSAAWAWDSSSS-
SSSSDDWAWDDDDWAAWVWASSSSAWASAWWWVWASAAAASSAWAWWWVWDSWVWV-
WAAWDDDDWAAAWDDDDWAWDWAAWVWASSAAWDWVWASSAWDWAAWWDSASDDSDS-
DSDWDDSSDDSAAAWASSAWASAAAAAAWVWVWASDDDDSAWVWAWDDWDSSSSAS-
DSDWVWVWVWDSWDSDWDWAAWVWASSAAAAASAWDWVWVWDSDDWVWAAASDWWD-
SASDDSD-
SAWVWVWVWAAWVWSDDDWVWDDWVWAAAAWDWAAWVWAAASDDDDSSDDSDWDSSS-
SAAAWDDWAAWASSAWVWVWAAASDSAASSDWDSDDSAAAASDDSDWDDSAASSSSSAW-
WAAAAAWDWAASAWDWDWDAASDSAAWVWASSAAWVWASSASSSDWVWVWDSASS-
SAASDWDSDDDSAAAWASSDDDSAAWVWASAWAAWDDWAAWVWASAWDWAWVWAWD-
WAWDDWVWVWDSWDSDWVWAAASDSAAAWAAWVWDSWDSSDDSSDWAAWDDDDDDSDS-
SAWASSSSDWVWDSDDWVWSDWVWAAAWDWAWDDDDSAAAASDSDWVWAWDDSDASD-
SAASDWDDWVWAAWDDDDWAAAWDDWVWVWASSAWVWAAWDDDDDDSSAWDWAAW-
WASDDSSSAAAWVWSDWVWDSDDWAAWDDSSSDWVWVWDSWDSSSAASDDWVWDSASDWV-
SAWASSAWVWV-
WAASDWDSASDDDDSDWAWDDWAAWDWAASSSAWVWASSAWVWDDDDWVWASSA-
WAAWASSAASDDSSAAAWDDWAAWDDWAAWDDSDWVWASAWVWDDSSSDWVWVWDS-
SDWVWVWDDWDSSSAWVWASSSDWVWDSAAASDWDDWVWVWDSASDSSSAASDWDDSSSS-
SSSSWASAWDDWVWVWASSSAWVWAWDWAAWDWAASAWASSDSAAASDDDSASSSAAS-
SAWVWVWVWASAAASASDDDDAWVWVWDSDDWVWSDWAAAAWDDSDDDWVWSAWVWASSAAAWV-
WASSSSSSDSSSVWDDSSAWDWVWAAWVWDSSSDWVWVWASSAWVWAAASDSAAWVWVW-
WAAWVWDSWDSSSDSSAWASSSDWDSAAASDSWAWDSSSAWDWAAWVWSDSSAWAWVW-
WAWDDWAAWASSASDSDSASASDWDSSSDWDDWVWDSSSAASSAWVWAAASDDDDWVWDSSS-
SAASAWVWASSDDSDSASDSSSAAWDWAWDDWASAWVWVWAAAAWVWDSWDSDWDD-
WAAWVWAWDWDASDDSSDWVWAWAWAWVWASSSAWVWASSDSSSAASDDSSDSAS-
SADWAAAWVWVWDSSSDWVWVWDSDDDDWAWAAWDDWVWDSSSSAADWAAAWVWAAWASS-
SSAAAAAAWAAWDDWVWVWAAWVWDSDDWVWSDWVWVWDSDDSAASAAASDDSDWVWDD-
WAAWVWASAWVWAAWDDWAWVWVWAWDDDSASDDSAASDDDSAAASDDSDSDSDW-
WAAWDDDDWAAWVWAWDWDSSDWAWVWASAWDWDWDDWVWVWDSSSAWDWVWVW-
SAASDSSDSAAASDSAAAWASSASDSSSSSAAWVWSDDDWVWVWDSWDWAAWDDSS-
SDSDWAWDDSSDWVWVWDDSSSWAASDDSAASAWDWAA-
SASDSASSAAAWDDWASSSAAWDWVWVWDSDDWVWSDWAWDDWAWDWAASASDSASSSSSS-
SDDDWVWSSSAWAAAWASSDDSAWVWSDSAASSSDDWAWDDSSSAASSAWVWVWVWAS-
SASDSASSDWDSSDSAAAWASAWVWVWASSAWAWVWDDWAAASSSAWVWVWDSDDWVWDS-
SDWVWVWAAASDDSD-
SAWAWAWVWASSAAWDWAASSSSASDSSDWVWVWDSDDSAASDSSDSDWAWVWDSSSSDWVWDS-
SDWVWDSDDDDDDDDDDDDSDAASDDSAASDSASDDSAAAWVWAWDWAWVWASSASDSAWAS-
DSDDSSSDSAASSAAWVWSDWAAWDDWAAWASAWAWDDWAAWVWSSDDSDWVWDS-
SASDDWVWDDWDAASAWASAWVWASSAAWVWAWDDDDWASAWDDASDSDSSASAAWDWW-
WASDSAWASAAAWASDSSDWVWVWSSAAAWASSDDWASAWDWAASAWVWDSWDWAAAS-
DSSAWASAWASSDDSAASAWVWASAWDDWAWDDSSDWDDWVWSDSAWVWVWVWASSAWVWASD-
DAWVWDDWAAWVWAAWDDSSDWVWDDWAAVWASAWVWSDWVWDSASAADWAWAWVW-
SASDSASDSAAWVWASSAAWAAASDSAAWVWVWVWASSSSAWAAASDDWVWDSDDWVWDDSDAAA-
SAASDDWVWDDWDSASAASDDSAASSASDWDWDDWAAWDDWVWVWDSWDSSAAASDDSDASD-
SAAWASSAWASSSAWVWASSDDSSSDWVWVWDSDDWVWVWDDDSAASSSDSASSSAWAAAWVW-
WAAWASSSASSSDSAWVWASDSAAASSDWAAAWDDSSDWAWVWDS-
SSAAASDDDDWAWDDWAWDDWAAWVWASSAWVWDAAWVWVWVWDSDDDDSAAS-
DSSDWDDDDDDWVWVWASSSAAAWAAWDDDDWAWVWDDWVWDDWASAWDWAAWASS-
SDWVWSDSSSSAWASSSAAWDWVWVWVWVWAAAWDDDDWVWSDWVWAAWASSAAWVWDAW-

ASSAWDWWDDWAAASSAWWASSAWDWWDDDDSDWWDWDSDSDWAWDWWDSDDW-
WAWDWWWWWWASSSS-
SASAWAAAWWWASSSAWWWWDDDDWDSSSAWASSDDDDWDDDDDSASDWASAASASSAA-
SAAWDDWAAWAAWDDDDSDWAAASAWDDDDWAAASSSAWDDSDWDDSDASDDSDWDS-
SASDSAAWWASAWASSSDWDSDDSAASASDDDDWWDWWDSSSASDSASAWWWAAASDD-
SAASSDWDSSDWDSSSDWWDSDWAWDWDSSDDSSAAASSSDSSDSAASDDSAASSAWW-
WASSDWAWDWWWWWWDSSASSDWDWWWWWAWDWAASAWASSWDDDSASDDWDS-
SAAAASDDWWDWWWWWWDSSSSSDWWDSS-
SSAWASAAWASDSASDDDSASDSAASDDDDSSDWWDSSDWWDSDWAAAWDDDDWAADSS-
SAAASSASDWWSAWWDWAWWDSSDWWDSSASSDWDDSDWDWAAAWWAWDDDD-
SAAASDDDDSDWAWWWWWWDSSSWASASDSAAWWAWASDSDSASAAAWDDWWDWASSAW-
WASSSSSAASDDDDWDDDDWDSWDSDSDSDSAAWAAAASDDDSAAAAAWDWASDDAWDD-
SAAASDDDDWAWDWWWWWWDSSASDDWDWAAS-
SAASSAWWWDSDWASAWDWAAWDDWDSSDDWWDSSAAAASDDDWDDSDASDDDDSSWA-
SAAAWDDWASAWWASSASDSSSSAWWDDWAAAAASSSDWAWDSDWDSSSDWDSSSSAW-
WASSSSDSSSWDSAAASDSWDSDWWDWWWDSDSASSSDWWDSSSSAWAAASDWWDSS-
SDSASSDWDDSDWWDWDDWAAASASDSAAWWAWDDWWDSSDDWAWWWWASSAW-
WAWDDDWDSSSSDWWDSSSDSASDSWDWAAAWDWAAWDDDSDDDWAWDDDDDDDD-
WAS-
DSDSAAWWWASAWAAASAWWDDASSSDWDDWDSSDWDSWDSDSAAASDSSDWAWDDDD-
SAASDDDWWDWAAWWASSAAWDWAAWWWWDSSDDDWAAAAWASSDDSAAWAAASAAA-
ASSSAWAAWWDSDWAWDSDWDSSDWWDSSDDSSSSAWDWAAWDDDDWAAAAAA-
WASSSSSSDSSSSSAASDDSAASASSAWASSAAWWDWDDDDWWDSDWDSDWAAAWDWA-
SAWWWASAAWDDWAASDSASSWDDSDWWDSDSDWWDSDSWDSAAASDSASAWWASSAA-
WASAWWASSSDDAWWDWWWDSSDWWDWWDSDSASDDDD-
DSASDDWWDWAWDDWAAWDDWWDWDDWASAWWASSSAWASSAWASSDDSSAAWDWA-
SAWASAAWDDSDWWDSDWAWDDWAAWASSSAASASSAAWDWAWWDDSAWASDDSS-
SDWWDSDWAAWDDWDSSSDSSSSDDDDSAASSSSAAASAWWWWWWASSAAWDWASS-
SSASDSASDSSSAADWAAAAAAWDDWAWWWWWWWDSDSAASS-
SDDSDSWAAAWDDWAAWDSDWAAAWAAWDDSDWAADSSAWASSAASDDWWDSSDD-
SAASDDWWDSDSASAAAASWASDDSAASDDDSAAASDSASDDWDDDDWDSDDSDASDSSASD-
SAAAAAAASDDDDDSAAAAAAAWDDWASSAWDASSSDWWDSDAASDDDDWDSWDWAAA-
SAWDDDDWAAAAWSAWASAWWASAAASAWDWDSDWAWDWDSSSDWDSSAADWWAS-
SAWWAWDDSDASDWWD-
SASDWWDWDDWAADSASSAASDSAAAWASAWWASSAAAASWASSDDDDSSASSAWWWASS-
SASDSAAAAASDDSSDDWAWDDSDASDDSDASDDWAWWDSDSASSAADWAAAWDWAASA-
WASAWAAWDSASSDWWDSDDDSDSDWWDSSSDWWDSDWWDWWWWWASDSASDDDDSS-
SASAWWDWAAAWASDSDWDSSAASAWASSAWWSDDDDWWDSDASDWDDDDDDWWD-
WAAASAWDWASAAAWWWWWWDSSDWDDSDASDSWWDSSSDWAAWDSSDDDDWA-
WASAWDWDSDWDDWAWDDDDSDWASAAWWDWAAWASDSAASSAWWWDD-
WASSAAWDWAAWWDWAWWWWDSSDSSSAASDDSDWAWDDWWDSSDWWDSDDDDDW-
WASAWWAWDWAAADSASDDSSSDSSSSSAAWWWASSSAWAWWDSDWAWASS-
SAWAAWWSDDWWDWWDSSDWWDSSSDSAAAWASDDSDWDDSDSAWDWASAWD-
WAAWDDASSSDWDSDSAASDDWAWDWWDSDAADWAASSSAWDDWAAWWAAWDDWAWDD-
WAAWASASDSSAAWASSSSAAWDWAWASSAWASASDDSAAAWASSDWDSSSSDWDSAA-
SAWWSDWDDWAAWDDASSDDSSAWAAAAWAAWDWASASSSSDWDSDDWAAWWDSDWDSS-
SSSAASDDSSAAWAAASDSAAASAWWDWAAWWDSSDWWDWASSAWASAWWASSSSSS-
DSSSAAWWWWWAWWAWDDSDDWAAWASAWDWWWDSSDDWAWAWDDDWAWWASS-
ASSSSSSAAWDWASDDDDDDSDWWDWWDSSDSSASDDSDSDSAWAAWASSAWWWASAWASS-
SSAWWAAWDDWAAWASSASDSAAAAAASDSSDDWAAWDDDDDDWDSAAAASDDSSA-
WASAAASDDSDWDSDDWAAWWDSDWWDSDWAWDSSDDWDSDDSDWDDWAAWDD-
WAAWAAASDSASSAWWAWDWAWDWDSSDWDSDDWAAWAAWDDWWWWWWDSD-
SASDWWDSDSDSDWAAWDDWWAAWD-

WAWDDDSASDSDWWDDWWWDSSSDWDSSDDSAWWWASSSAWASSSAWASSSAWASS-
 SDDWAWDDWWDDSSDWDWWDDWWWDSSAASDDSSSSSDWWAWDWAWWWDDAS-
 DSSSSSSAASDDSSAWAAWWWASSSDSSDWDSSAASDDSSSAWAAASDSASDSDWDSS-
 SAAAASDSASDDDWAWDDSSSSASDSSAAWDWAWDWAWDDWASAASSAWWWAWD-
 WAWDWDSWDWDDWAASAWASAWASSSDSSSSDSSASDDDSAAAAASDSASDDDDSS-
 SAAWDWAASAAASDDSAWWASDDSDDDDDWASAAAWDDDDWAAWDWDDDDDDDDSSDWW-
 WASDSAWASDSAWAASDSDWWDDSSASDSAAAASDDDDSDWDDDDSDSDSSAWAA-
 WAAWWDSDDDWAASAWAWWASDSAAWDWDDWASSAASAWWWA-
 SAWWDDDDSSDDDDWAAWDDWAAAAAASASDSASDDDDDSAAAASDSAAAWWASDDSAASDS-
 SAWASSSDWWDDDDSSWASAAAWDDDDDDWASDSAWASDDSAWWAAAAASDSSSAAWDWAW-
 WAAASDDSSSSDDDDDDDDDDWAAAAAWDSWDWAWDDSSSAWASSSDWWDDSDWWDDSDASD-
 SAAASDWDDDDDWWWWAAWDDSSS-
 SAWDWASAWASAAAWWAAASDDSAASDDDDDDDDDDDDDDDDDDWWDDDSAAASDDDDDD-
 WASAAAWDDDDWDWDAASSAWDDWAASDSAAAWDDDDSDWAWAAASDDSAASDSAAW-
 WAAASDDSAAAAAWDDWAAWDDDDWASAWWWAASDSSAWAAAAWDDDDWAAWWAAS-
 DSAAWDWAWWWDDSAASDDDDSSDWWDDSDWWDSSDDSAASDDSDSASDDWWDDWAWA-
 SAWWDDWDDDDWAAAAWASDSAAAAWDSWDWAWDDDDSDDDWAWAWWAWASSDWWWSAA-
 WASAWWWDSWDWAWDDSDWDDWAAAWDDDDWWDSSDSASDDDDSAAAAAASDSSSDSDWD-
 WAAWDDDDSDASDSASASSAWWWASASDSSDDDSAAASA-
 WAAASDDSAASDDWDSDDDDWAAWDDDDDDWAWDWAWASSAWWAWDWWDSDWDDSDASDSS-
 SSSAAAASDDDDSAASDDSAASDSAAWWASSAASDDDDDD

filename: labyrinth5.txt

Hier spielt die Wipeoutregel eine wichtige Rolle. Durch die Wipeoutregel werden vor allem Wege verworfen, bei denen beide Charaktere mind. einmal in eine Grube gefallen sind. Dadurch wurde die benötigte Zeit ebenfalls verringert, um es zu berechnen.

Hinzu kommt noch, dass für ProgressPuffer = 21 die nächstgelegene Best-Lösung (1322 Aktionen) in 2.716236114501953s berechnet wird. Weitere Informationen sind in der Datei „labyrinth5_besteausgabe.txt“ zu finden.

Amount of Actions: 1329; Required time: 0.5536153316497803

DDSSDDWDWDDSAASSAASDDSAASDSDSAAASDSASDSSASSDSDSDWSSSASDDSDSSSS-
 SAASDDSDDDDDWDSSSSA-
 ASSSSDSSASSSDSDDDWWDWDDSWDDSDWDDWDSWDDWDDSSSSDSSSSSDSDWDS-
 DSDSSSAAAAASADDSAAASDDSSSDWWSDWDDSSSSASAASDDSSDDSDSDSDWDDDD-
 WAAWDDWDDWAAAWWWWWAWWDSDDDW-
 WAAAAWWDWDDWDDWWWWDDSSDWDWWDSDWDSDDDDSDSSASSAASASSDSSAASDS-
 DSDWDSDDSSAS-
 SAASDDSSASSSDWWDWWDSDDDWDDWDDWWWDSDSDSDWDDSAASSDDWDDWWWWDDSS-
 DSDDDWDDSDSDWWAWASSAWWDWDDSDSDWDDWWWDSSDWWDDDDSDSDSSA-
 ASSDSSDSSSAAADSASAAAWAWAASAASSAASAS-
 DSAASSASDDDSAAASDSDSSASSDWDSDDDSDSDSDSDSDSDWDDDDSDSDWDDDDWAAAA-
 SAAWASAAAAAWAWDWDDDDDDWDDSDSSWDSASDSAAASAASDSSSSASDSDSSAS-
 SAWASSDSSSSSDWWDSDSSSDSDSAAASAWAASASSAASDSDSSWDSDDSAASDSSS-
 DSDWDSDDWWDWAWDDDDDDWAWWAWDDWDDSDWDDSDSDSDSDSDSDSDSDSDSD-
 DSSDSDWDWAWWDWDDWWDWSDWWDWDAWDDSSSDSSAWASSASDDSDSDWDDSDASS-
 SASDSDSDSDSDSDSDSASSSDSAAWASASAAASAWAWAASAWWAAASAAWWWWAWASS-
 SDWAAADSADSSAASAAAWWASDDSAASASASSASDSSSDSDSDSDSDSDSDSDSDSDSD-
 DSDWWDWAAASASASAAWAWAASDDDDWDDWDDSDSDWDDDDWWDWWSAAAAA-
 WAAWWWWAAWAAWAAWSDWDSDDAASDDSDSDSDWDDSDSDSDSDSDSDSDSDSDSD-
 SASASASAAADSASDSDSSAASDDSDSSSAAASAASSAASSAWASSAAWWAASSSDSDSS-
 ASSDWDDSDSDSDSDWDDWAWDDDDSDSDSDSDSDSDSDSDSDSDSDSDSDSDSDSDSD-
 ASSDWDDSD

Wenn es in beiden Labyrinthen keine Möglichkeit zum Ziel gibt, so gibt dieses Programm dem Nutzer Bescheid, dass in beiden keine Lösung gefunden wurde. Hier wird wieder deutlich, wie viel Zeit gespart wird, wenn man alle Labyrinth zunächst einmal einzeln löst.

Dateiname: EdgeCase_1.txt

Ausgabe: No solutions for both.

n-Labyrinth: Custom_n-lab1.txt

Für das erste n-Labyrinth habe ich mir $k = 3$ (3 Labyrinth) ausgewählt. Dafür habe ich die Eingabe aus der labyrinth1.txt genommen und ein weiteres Labyrinth hinzugefügt. Somit ergibt es sich ein 6-dimensionales Labyrinth, das alle gleichzeitig verlaufen. Aus Kontrollgründen habe ich die Lösungen der einzelnen Labyrinth ebenfalls ausdrucken lassen. Was hier aber auffällt ist, dass es tatsächlich die gleiche Lösung ist wie die von labyrinth1.txt, aber das liegt daran, dass das neue Labyrinth ebenfalls mit der Lösung gelöst werden kann.

filename: Custom_n-lab1.txt, Required time: 0.0170137882232666

SSSDWSDWWAAWDDDDSSS

DDDDSAWAASSDWDSDWDS

DDDDSSAWASAWASSDDDD

Amount of Actions: 31

DDDDSSAWASAWASSDWDSDWWAAWDDDDSSS

n-Labyrinth: Custom_n-lab2.txt

Für das zweite n-Labyrinth wollte ich mir ein größeres Labyrinth aussuchen. Dafür habe ich Labyrinth 2 genommen, es dupliziert und das Duplikat an der x-Achse sowie y-Achse spiegeln lassen. Es gibt daher 4 Labyrinth.

filename: Custom_n-lab2.txt, Required time: 1.749234676361084

DDSSDDDDWDDDSASDSSASAWAAAAWAASSSDSDDDSSDDWWDSDS

SSDDSSDSAASASDSDSDWWDWDDDDSSASASSDDS

SDSDWWDDSSDDDDSDSSSAWAAAAWASASSDSASDDDDWDDSSDD

SDDSSASASDDDDWWDWWDSDSDSASAASDSSDDSS

Amount of Actions: 84

SSWDSDSSDDSAASASDSDDDSDWWDWWDSDSDDDDSASDSSSASAWAAAAWASASS-
DSASDSDDDSSDDWWDSDSDSD

n-Labyrinth: Custom_n-lab3-failure.txt

Für das Letzte habe ich dasselbe wie bei Custom_n-lab2.txt genommen, nur diesmal eine Grube so verschoben, dass sich eines der duplizierten Labyrinth nicht lösen lässt. Statt der Lösung wird deshalb für Labyrinth 4 (es fängt bei 0 an zu zählen) der Wert "None" angezeigt und für die parallelen Berechnungen ignoriert.

filename: Custom_n-lab3-failure.txt, Required time: 0.49280595779418945

DDSSDDDDWDDDSASDSSASAWAAAAWAASSSDSDDDSSDDWWDSDS

SSDDSSDSAASASDSDSDWWDWDDDDSSASASSDDS

SDSDWWDDSSDDDDSDSSSAWAAAAWASASSDSASDDDDWDDSSDD

None

Labyrinth 3 is not solveable.

Amount of Actions: 78

DDSDSDDDWDDDDSSDDDDSDDSASDSSASAWAAAWASASSSDWDSDDDD-
SAASDDWWWDWDDDDSDDSASASSDDS

6 Quellcode

```
def read_labyrinth(content):
    # Text vorbereiten
    lines = content.strip().split("\n")
    line = 0

    # Labyrinthgröße ablesen
    n, m = map(int, lines[line].split())
    line += 1

    # Labyrinth 1:
    # Wände ablesen
    vertical_walls_1 = [list(map(int, lines[line + i].split())) for i in range(m)]
    line += m

    horizontal_walls_1 = [list(map(int, lines[line + i].split())) for i in range(m - 1)]
    line += m - 1

    # Anzahl Gruben ablesen
    num_pits_1 = int(lines[line])
    line += 1

    # Gruben ablesen
    pits_1 = []
    for i in range(num_pits_1):
        x, y = map(int, lines[line].split())
        pits_1.append((x, y))
        line += 1

    # Labyrinth 2:
    # Wände ablesen
    vertical_walls_2 = [list(map(int, lines[line + i].split())) for i in range(m)]
    line += m

    horizontal_walls_2 = [list(map(int, lines[line + i].split())) for i in range(m - 1)]
    line += m - 1

    # Anzahl Gruben ablesen
    num_pits_2 = int(lines[line])
    line += 1

    # Gruben ablesen
    pits_2 = []
    for i in range(num_pits_2):
        x, y = map(int, lines[line].split())
        pits_2.append((x, y))
        line += 1

    # Zur Dictionary umwandeln und zurückgeben.
    return {
        "labyrinth_1": {
            "vertical_walls": vertical_walls_1,
            "horizontal_walls": horizontal_walls_1,
            "pits": set(pits_1),
        },
        "labyrinth_2": {
```

```

        "vertical_walls": vertical_walls_2,
        "horizontal_walls": horizontal_walls_2,
        "pits": set(pits_2),
    },
    "dimensions": (n, m),
}

def solve_labyrinth(dimensions, vertical_walls, horizontal_walls, pits):
    """
    solve labyrinth: Berechnet den Weg eines einzelnen Labyrinth.
    """
    n, m = dimensions
    start = (0, 0) # (x, y)
    goal = (n - 1, m - 1)
    directions = [(1, 0), (0, 1), (-1, 0), (0, -1)] # Rechts, Unten, Links, Oben
    visited = set(pits) # Besuchte Koordinaten (startet mit den Gruben, um alle Gruben zu
    vermeiden)
    payload = {
        "position": start, # (0, 0)
        "movement": "", # Bewegungssymbol
        "before": None, # vorherige payload um vorherige Bewegungen rekursiv aufzulisten
    }
    queue = [payload] # Warteschlange

    while queue:
        payload = queue.pop(0)
        (x, y) = payload['position']

        # Für alle 4 Richtungen probieren
        for dx, dy in directions:
            nx, ny = x + dx, y + dy

            # innerhalb des Spielbereichs und noch nicht besucht
            if not 0 <= nx < n or not 0 <= ny < m or (nx, ny) in visited:
                continue

            if dy == 1 and horizontal_walls[y][x] == 1: # Unten
                continue
            if dx == 1 and vertical_walls[y][x] == 1: # Rechts
                continue
            if dy == -1 and horizontal_walls[y - 1][x] == 1: # Oben
                continue
            if dx == -1 and vertical_walls[y][x - 1] == 1: # Links
                continue

            # Bewegungssymbol bestimmen
            if dy == 1:
                movementletter = "S"
            if dy == -1:
                movementletter = "W"
            if dx == 1:
                movementletter = "D"
            if dx == -1:
                movementletter = "A"

            if (nx, ny) == goal:
                # Rekursiv durch die Befragung aller vorherigen payloads die benötigte
                # Bewegung sowie Koordinatenliste bestimmen
                pastPositions = [(nx, ny)]
                while True:
                    movementletter = payload['movement'] + movementletter # Bewegung
                    pastPositions.insert(0, payload['position']) # Koordinatenliste

```

```

        if payload["before"] is None:
            return pastPositions, movementletter
        payload = payload['before'] # zur vorherigen payload bzw. Koordinate
        zurückgehen

        visited.add((nx, ny)) # Neue Koordinaten als besucht markieren
        newPayload = {
            "position": (nx, ny),
            "movement": movementletter,
            "before": payload,
        }
        queue.append(newPayload) # Neue Koordinaten für weitere Berechnungen in die
        Warteschlange hinzufügen.

        return None, None # Bei keiner Lösung

def solve_duo_labyrinth(dimensions, labyrinth1, labyrinth2, solution1, solution2,
    progressPuffer = 10):
    """
    solve duo labyrinth: Berechnet Weg in zwei Labyrinthen gleichzeitig (4-Dimensionale
    Labyrinth).
    """
    n, m = dimensions
    goal = (n - 1, m - 1)
    directions = [(1, 0), (0, 1), (-1, 0), (0, -1)] # Rechts, Unten, Links, Oben
    best = 0 # Rekordpunktzahl
    payload = {
        "progress1": 0, # Punktzahl für Labyrinth 1
        "progress2": 0, # Punktzahl für Labyrinth 2
        "position1": (0, 0), # Koordinate Labyrinth 1 (x, y)
        "position2": (0, 0), # Koordinate Labyrinth 2 (x, y)
        "movement": "", # Bewegungssymbol
        "wipeouts": (False, False), # vgl. Wipeoutsregel
        "before": None, # vorherige payload um vorherige Bewegungen rekursiv aufzulisten
    }
    visited = set() # Besuchte Koordinaten
    queue = [payload]
    while queue:
        payload = queue.pop(0)
        # Für alle 4 Richtungen
        for dx, dy in directions:
            p1 = payload["position1"]
            p2 = payload["position2"]
            np1 = None
            np2 = None
            NewWipeouts = payload['wipeouts']
            # Bewegung nach rechts
            if dx == 1:
                if (p1[0] + 1) != n and labyrinth1["vertical_walls"][p1[1]][p1[0]] == 0:
                    np1 = (p1[0] + dx, p1[1] + dy)
                if (p2[0] + 1) != n and labyrinth2["vertical_walls"][p2[1]][p2[0]] == 0:
                    np2 = (p2[0] + dx, p2[1] + dy)
            # Bewegung nach links
            if dx == -1:
                if (p1[0] - 1) >= 0 and labyrinth1["vertical_walls"][p1[1]][p1[0] - 1] == 0:
                    np1 = (p1[0] + dx, p1[1] + dy)
                if (p2[0] - 1) >= 0 and labyrinth2["vertical_walls"][p2[1]][p2[0] - 1] == 0:
                    np2 = (p2[0] + dx, p2[1] + dy)
            # Bewegung nach unten
            if dy == 1:
                if (p1[1] + 1) != m and labyrinth1["horizontal_walls"][p1[1]][p1[0]] == 0:
                    np1 = (p1[0] + dx, p1[1] + dy)

```

```

        if (p2[1] + 1) != m and labyrinth2["horizontal_walls"][p2[1]][p2[0]] == 0:
            np2 = (p2[0] + dx, p2[1] + dy)
# Bewegung nach oben
if dy == -1:
    if (p1[1] - 1) >= 0 and labyrinth1["horizontal_walls"][p1[1] - 1][p1[0]] == 0:
        np1 = (p1[0] + dx, p1[1] + dy)
    if (p2[1] - 1) >= 0 and labyrinth2["horizontal_walls"][p2[1] - 1][p2[0]] == 0:
        np2 = (p2[0] + dx, p2[1] + dy)

# Falls ein Charakter am Ziel angekommen ist -> nicht wegbewegen
if p1 == goal:
    np1 = p1
if p2 == goal:
    np2 = p2

if np1 is None:
    np1 = p1
if np2 is None:
    np2 = p2

# Falls Charakter auf Gruben -> zurück zum Startpunkt
if np1 in labyrinth1["pits"]:
    np1 = (0, 0)
    if NewWipeouts[1] == True: # Falls der Charakter im Labyrinth 2 schon mal in
eine Grube gefallen ist -> Bewegung verwerfen
        continue
    NewWipeouts = (True, False) # Der Charakter im Labyrinth 2 darf nicht mehr in
eine Grube fallen.
if np2 in labyrinth2["pits"]:
    np2 = (0, 0)
    if NewWipeouts[0] == True: # Falls der Charakter im Labyrinth 1 schon mal in
eine Grube gefallen ist -> Bewegung verwerfen
        continue
    NewWipeouts = (False, True) # Der Charakter im Labyrinth 1 darf nicht mehr in
eine Grube fallen.

# Schon bereits besucht? -> Bewegung verwerfen
if (np1, np2) in visited:
    continue

# Bewegungssymbol bestimmen
if dy == 1:
    movementletter = "S"
if dy == -1:
    movementletter = "W"
if dx == 1:
    movementletter = "D"
if dx == -1:
    movementletter = "A"

# Falls beide Charaktere am Ziel angekommen sind.
if np1 == goal and np2 == goal:
    while True:
        # Bewegungssymbol rekursiv wiederherstellen, indem man die Bewegungen von
vorherigen Stellungen anschaut
        movementletter = payload["movement"] + movementletter
        if payload['before'] is None: # Falls keine weiteren vorherigen Stellungen
gibt -> Bewegungen zurückgeben.
            return movementletter
        payload = payload['before'] # Zur vorherigen Stellung zurückgehen.

# Punktzahlkontrolle: Labyrinth 1

```

```

    if solution1[payload['progress1'] - 1] == np1: # Ist der Charakter zurückgegangen?
-> Punktabzug
        newProgress1 = payload['progress1'] - 1
    elif payload['progress1'] + 2 != len(solution1) and solution1[payload['progress1']
+ 1] == np1: # Ist der Charakter vorwärts gegangen? -> Punktbewohnung
        newProgress1 = payload['progress1'] + 1
    else:
        newProgress1 = payload['progress1'] # Keine Veränderung bzw. Punkte
zurücksetzen wenn der Charakter wieder beim Startpunkt befindet.
        if np1 == (0, 0):
            newProgress1 = 0
    # Punktzahlkontrolle: Labyrinth 2
    if solution2[payload['progress2'] - 1] == np2:
        newProgress2 = payload['progress2'] - 1
    elif payload['progress2'] + 2 != len(solution2) and solution2[payload['progress2']
+ 1] == np2:
        newProgress2 = payload['progress2'] + 1
    else:
        newProgress2 = payload['progress2']
        if np2 == (0, 0):
            newProgress2 = 0

    # Fortschrittskontrolle: Mind. 1 Labyrinth muss 1 Punkt bekommen haben.
    if newProgress1 <= payload["progress1"] and newProgress2 <= payload['progress2']:
        continue

    # Punktzahlkontrolle: Ist die aktuelle Punktzahl zu niedrig?
    if (newProgress1 + newProgress2 + progressPuffer) < best:
        continue

    # Bei neuer Rekordpunktzahl -> als Rekord speichern
    if (newProgress1 + newProgress2) > best:
        best = newProgress1 + newProgress2

    newPayload = {
        "progress1":newProgress1,
        "progress2":newProgress2,
        "position1":np1,
        "position2":np2,
        "movement":movementletter,
        "wipeouts":NewWipeouts,
        "before": payload,
    }
    visited.add((np1, np2)) # Stellung als besucht markieren
    queue.append(newPayload)

    # Bei keiner Lösung müssen die Kriterien für die Punktzahlkontrolle niedriger gestellt
werden, um alternative Wege auszuprobieren.
    # Diese Zeile wurde bei keiner Beispielsaufgabe verwendet. Trotzdem als Failsafe
implementiert, schließlich muss es eine Lösung geben.
    return solve_duo_labyrinth(dimensions, labyrinth1, labyrinth2, solution1, solution2,
progressPuffer + 5)

def main(filename):
    # Beispielsaufgabe lesen
    with open("Beispiele\\" + filename, 'r') as f:
        content = f.read()

    # Labyrinth als Dictionary laden
    labyrinths = read_labyrinth(content)
    print(f"filename: {filename}")
    paths = [] # Positionenliste

```



```

movements = [] # Bewegungsliste
for key in ["labyrinth_1", "labyrinth_2"]:
    labyrinth = labyrinths[key]
    path, movement = solve_labyrinth(
        labyrinths["dimensions"],
        labyrinth["vertical_walls"],
        labyrinth["horizontal_walls"],
        labyrinth["pits"],
    )
    paths.append(path)
    movements.append(movement)

# Kontrolle, ob alle Labyrinth sich lösen lassen.
if (paths[0] is None) and (paths[1] is None):
    print("No solutions for both.")
    return
if (paths[0] is None):
    print("No solution for labyrinth 1")
    print(f"Amount of Actions: {len(movements[1])}")
    print(movements[1])
    return
if (paths[1] is None):
    print("No solution for labyrinth 2")
    print(f"Amount of Actions: {len(movements[0])}")
    print(movements[0])
    return

# Beide Labyrinth unter parallelen Bedingungen lösen
movement = solve_duo_labyrinth(
    labyrinths["dimensions"],
    labyrinths["labyrinth_1"],
    labyrinths["labyrinth_2"],
    paths[0],
    paths[1],
)
print(f"Amount of Actions: {len(movement)}")
for i in movement:
    print(i, end=" ")
print()

```

6.1 Huffman-Codierung

```

def read_labyrinth(content):
    # Text vorbereiten
    lines = content.strip().split("\n")
    line = 0

    # Labyrinthgröße ablesen
    try:
        n, m, nlab = map(int, lines[line].split())
    except ValueError:
        n, m = map(int, lines[line].split())
        nlab = 2
    line += 1

    dimensions = (n, m)
    labyrinths = []
    for i in range(nlab):
        # Wände ablesen
        vertical_walls = [list(map(int, lines[line + j].split())) for j in range(m)]
        line += m

        horizontal_walls = [list(map(int, lines[line + j].split())) for j in range(m - 1)]
    
```

```

    line += m - 1

    # Anzahl Gruben ablesen
    num_pits = int(lines[line])
    line += 1

    # Gruben ablesen
    pits = []
    for j in range(num_pits):
        x, y = map(int, lines[line].split())
        pits.append((x, y))
        line += 1
    labyrinths.append(
        {
            "vertical_walls": vertical_walls,
            "horizontal_walls": horizontal_walls,
            "pits": set(pits),
        }
    )

    #print(labyrinths)
    # Zur Dictionary umwandeln und zurückgeben.
    return dimensions, labyrinths

def solve_labyrinth(dimensions, vertical_walls, horizontal_walls, pits):
    """
    solve labyrinth: Berechnet den Weg eines einzelnen Labyrinths.
    """
    n, m = dimensions
    start = (0, 0) # (x, y)
    goal = (n - 1, m - 1)
    directions = [(1, 0), (0, 1), (-1, 0), (0, -1)] # Rechts, Unten, Links, Oben
    visited = set(pits) # Besuchte Koordinaten (startet mit den Gruben, um alle Gruben zu vermeiden)
    payload = {
        "position": start, # (0, 0)
        "movement": "", # Bewegungssymbol
        "before": None, # vorherige payload um vorherige Bewegungen rekursiv aufzulisten
    }
    queue = [payload] # Warteschlange

    while queue:
        payload = queue.pop(0)
        (x, y) = payload['position']

        # Für alle 4 Richtungen probieren
        for dx, dy in directions:
            nx, ny = x + dx, y + dy

            # innerhalb des Spielbereichs und noch nicht besucht
            if not 0 <= nx < n or not 0 <= ny < m or (nx, ny) in visited:
                continue

            if dy == 1 and horizontal_walls[y][x] == 1: # Unten
                continue
            if dx == 1 and vertical_walls[y][x] == 1: # Rechts
                continue
            if dy == -1 and horizontal_walls[y - 1][x] == 1: # Oben
                continue
            if dx == -1 and vertical_walls[y][x - 1] == 1: # Links
                continue

```

```

# Bewegungssymbol bestimmen
if dy == 1:
    movementletter = "S"
if dy == -1:
    movementletter = "W"
if dx == 1:
    movementletter = "D"
if dx == -1:
    movementletter = "A"

if (nx, ny) == goal:
    # Rekursiv durch die Befragung aller vorherigen payloads die benötigte
    # Bewegung sowie Koordinatenliste bestimmen
    pastPositions = [(nx, ny)]
    while True:
        movementletter = payload['movement'] + movementletter # Bewegung
        pastPositions.insert(0, payload['position']) # Koordinatenliste
        if payload["before"] is None:
            return pastPositions, movementletter
        payload = payload['before'] # zur vorherigen payload bzw. Koordinate
        zurückgehen

    visited.add((nx, ny)) # Neue Koordinaten als besucht markieren
    newPayload = {
        "position": (nx, ny),
        "movement": movementletter,
        "before": payload,
    }
    queue.append(newPayload) # Neue Koordinaten für weitere Berechnungen in die
    Warteschlange hinzufügen.

    return None, None # Bei keiner Lösung

def solve_n_labyrinth(dimensions, labyrinths, solutions, progressPuffer = 11):
    """
    solve n labyrinth: Berechnet Weg in n Labyrinth gleichzeitig (2*n-dimensionale
    Labyrinth).
    """
    n, m = dimensions
    goal = (n - 1, m - 1)
    directions = [(1, 0), (0, 1), (-1, 0), (0, -1)] # Rechts, Unten, Links, Oben
    best = 0 # Rekordpunktzahl
    nLabyrinths = len(labyrinths)
    payload = {
        "progress": [0 for _ in range(nLabyrinths)], # Punktzahl
        "positions": [(0, 0) for _ in range(nLabyrinths)], # Koordinate Labyrinth
        "movements": "", # Bewegungssymbol
        "wipeouts": (False,) * nLabyrinths, # vgl. Wipeoutsregel
        "before": None, # vorherige payload um vorherige Bewegungen rekursiv aufzulisten
    }
    visited = set() # Besuchte Koordinaten
    queue = [payload]
    while queue:
        payload = queue.pop(0)
        #print(payload)
        for dx, dy in directions:
            nps = []
            cancel = False
            newWipeouts = payload['wipeouts']
            for LabId in range(nLabyrinths):
                p = payload["positions"][LabId]
                np = None

```

```

# Bewegung nach rechts
if dx == 1:
    if (p[0] + 1) != n and labyrinths[LabId]["vertical_walls"][p[1]][p[0]] ==
0:
        np = (p[0] + dx, p[1] + dy)
# Bewegung nach links
if dx == -1:
    if (p[0] - 1) >= 0 and labyrinths[LabId]["vertical_walls"][p[1]][p[0] - 1]
== 0:
        np = (p[0] + dx, p[1] + dy)
# Bewegung nach unten
if dy == 1:
    if (p[1] + 1) != m and labyrinths[LabId]["horizontal_walls"][p[1]][p[0]]
== 0:
        np = (p[0] + dx, p[1] + dy)
# Bewegung nach oben
if dy == -1:
    if (p[1] - 1) >= 0 and labyrinths[LabId]["horizontal_walls"][p[1] - 1]
[p[0]] == 0:
        np = (p[0] + dx, p[1] + dy)

# Falls ein Charakter am Ziel angekommen ist -> nicht wegbewegen
if p == goal:
    np = p

if np is None:
    np = p

# Falls Charakter auf Gruben -> zurück zum Startpunkt
if np in labyrinths[LabId]["pits"]:
    np = (0, 0)
    newWipeouts = newWipeouts[:LabId] + (True,) + newWipeouts[LabId + 1:]
    if not (False in newWipeouts): # Falls alle Charakter schon mal in eine
Grube gefallen ist -> Bewegung verwerfen
        cancel = True
        break

nps.append(np)

if cancel:
    #print("cancel called")
    continue
# Schon bereits besucht? -> Bewegung verwerfen
if tuple(nps) in visited:
    #print(visited)
    #print("visited failed")
    continue

# Bewegungssymbol bestimmen
if dy == 1:
    movementletter = "S"
if dy == -1:
    movementletter = "W"
if dx == 1:
    movementletter = "D"
if dx == -1:
    movementletter = "A"

# Falls alle Charaktere am Ziel angekommen sind.
#if not (False in (i == goal for i in nps)):
if not (False in (i == goal for i in nps)):
    while True:

```

```

        # Bewegungssymbol rekursiv wiederherstellen, indem man die Bewegungen von
        # vorherigen Stellungen anschaut
        movementletter = payload["movements"] + movementletter
        if payload['before'] is None: # Falls keine weiteren vorherigen Stellungen
            # gibt -> Bewegungen zurückgeben.
            return movementletter
        payload = payload['before'] # Zur vorherigen Stellung zurückgehen.

    # Punktzahlkontrolle
    newProgressList = []
    for nLab in range(nLabyrinths):
        if solutions[nLab][payload['progress'][nLab] - 1] == nps[nLab]: # Ist der
            # Charakter zurückgegangen? -> Punktabzug
            newProgress = payload['progress'][nLab] - 1
        elif payload['progress'][nLab] + 2 != len(solutions[nLab]) and solutions[nLab]
            # [payload['progress'][nLab] + 1] == nps[nLab]: # Ist der Charakter vorwärtsgegangen? ->
            # Punktbelohnung
            newProgress = payload['progress'][nLab] + 1
        else:
            newProgress = payload['progress'][nLab] # Keine Veränderung bzw. Punkte
            # zurücksetzen wenn der Charakter wieder beim Startpunkt befindet.
            if nps[nLab] == (0, 0):
                newProgress = 0
            newProgressList.append(newProgress)

    # Fortschrittskontrolle: Mind. 1 Labyrinth muss 1 Punkt bekommen haben.
    if not (True in (payload['progress'][nLab] <= newProgressList[nLab] for nLab in
        range(nLabyrinths))):
        #print("Failed progression")
        continue

    # Punktzahlkontrolle: Ist die aktuelle Punktzahl zu niedrig?
    if (sum(newProgressList) + progressPuffer) < best:
        #print("Missed highscore")
        continue

    # Bei neuer Rekordpunktzahl -> als Rekord speichern
    if sum(newProgressList) > best:
        best = sum(newProgressList)

    newPayload = {
        "progress":newProgressList, # Punktzahl
        "positions": nps, # Koordinate Labyrinth
        "movements": movementletter, # Bewegungssymbol
        "wipeouts": newWipeouts, # vgl. Wipeoutsregel
        "before": payload, # vorherige payload um vorherigen Bewegungen rekursiv
        # aufzulisten
    }
    visited.add(tuple(nps)) # Stellung als besucht markieren
    queue.append(newPayload)

    # Bei keiner Lösung müssen die Kriterien für die Punktzahlkontrolle niedriger gestellt
    # werden, um alternative Wege auszuprobieren.
    # Diese Zeile wurde bei keiner Beispielsaufgabe verwendet. Trotzdem als Failsafe
    # implementiert, schließlich muss es eine Lösung geben.
    #return solve_n_labyrinth(dimensions, labyrinths, solutions, progressPuffer + 5)
    return ""
def main(filename):
    # Beispielsaufgabe lesen
    with open("Beispiele\\" + filename, 'r') as f:
        content = f.read()

```

```
# Labyrinth als Dictionary laden
dimensions, labyrinths = read_labyrinth(content)
print(f"filename: {filename}")
paths = [] # Positionenliste
movements = [] # Bewegungsliste
for i in range(len(labyrinths)):
    labyrinth = labyrinths[i]
    path, movement = solve_labyrinth(
        dimensions,
        labyrinth["vertical_walls"],
        labyrinth["horizontal_walls"],
        labyrinth["pits"],
    )
    paths.append(path)
    movements.append(movement)
print(movement)

# Kontrolle, ob alle Labyrinth sich lösen lassen.
newLabyrinths = []
newPaths = []
for i, path in enumerate(paths):
    if path is None:
        print(f"Labyrinth {i} is not solveable.")
    else:
        newLabyrinths.append(labyrinths[i])
        newPaths.append(path)
if len(labyrinths) == 1:
    print("There is only one labyrinth to solve.")
if len(labyrinths) == 0:
    print("There is no labyrinth to solve.")
return

# Alle Labyrinth unter parallelen Bedingungen lösen
movement = solve_n_labyrinth(
    dimensions,
    newLabyrinths,
    newPaths,
)
print(f"Amount of Actions: {len(movement)}")
for i in movement:
    print(i, end=" ")
print()
```